

VOLTEGE ELECTRONICS

Project Documentation

ETL · data model · DAX · report · analysis walkthrough

Fictional company · synthetic seeded data

01 — Business Context & KPI Framework

This document defines *the business* the analytics are built for: who VoltEdge Electronics is, how it makes money, who consumes the report, and the exact definition of every metric. Everything downstream (data model, DAX, report pages) traces back to this document.

1. Company profile

Attribute	Value
Name	VoltEdge Electronics
Type	Pure-play online retailer (direct-to-consumer), no physical stores
Category	Consumer electronics
Founded	2022 (US); trading history predates the analytics extract
HQ	Austin, TX, USA
Reporting currency	USD
Fulfillment	Company-operated fulfillment centers + regional 3PL partners
Analytics extract window	2024-07-01 → 2026-06-30 (24 months)

Why 24 months

Two complete trailing years lets every metric be shown **year-over-year against the same period last year**:

Period	Range
Current year (CY)	2025-07-01 → 2026-06-30
Prior year (PY)	2024-07-01 → 2025-06-30

Market expansion timeline

VoltEdge scaled from one market to four *during the extract window*. The report must make this legible — total revenue “steps up” as markets go live, and blended metrics (CAC, AOV, margin, delivery time) shift as the market mix changes.

Market	Currency	Live since	In the extract	Fulfillment centers	Primary carriers
United States	USD	2022	full 24 months	Dallas TX, Reno NV	UPS, FedEx, USPS
United Kingdom	GBP	2025-01-01	18 months	Birmingham	Royal Mail, DPD, Evri
Germany	EUR	2025-01-01	18 months	Leipzig	DHL, Hermes, DPD
Brazil	BRL	2025-07-01	12 months	São Paulo (Cajamar)	Correios, Jadlog, Loggi

Product catalog

~300 active SKUs organized as **Category** → **Subcategory** → **Brand** → **Product**.

Category	Example subcategories	Typical gross margin
Smartphones	Flagship, Mid-range, Budget	8–14%
Laptops & Tablets	Ultrabooks, Gaming laptops, Tablets	10–16%
Audio	Headphones, Earbuds, Speakers, Soundbars	22–35%
Gaming	Consoles, Controllers, VR, Games	6–18%
Smart Home	Hubs, Cameras, Lighting, Thermostats	20–30%
Wearables	Smartwatches, Fitness bands	18–28%
Accessories	Cables, Chargers, Cases, Storage, Power banks	35–55%

Why this matters analytically: electronics is a **thin-margin** business. Flagship phones and consoles drive revenue but barely any profit; accessories and audio carry the margin. A recurring executive question is “*are we growing revenue or growing profit?*” — the answer depends on **category mix**, which is why price–volume–mix analysis is a first-class citizen in this report.

2. Business model & economics

Revenue streams

- 1. **Product sales** — the vast majority of revenue.
- 2. **Shipping fees** — charged on orders below a free-shipping threshold (varies by market).
- 3. **Extended warranty / protection plans** — attach-rate add-on, high margin, modeled as a separate line item (`line_type = 'warranty'`) on a share of eligible orders.

Cost structure (modeled)

- **COGS** — unit cost × quantity (per SKU, sourced from the purchase price of the stock actually sold).
- **Discounts** — promotional and coupon reductions (campaign-driven and site-wide sales).
- **Returns** — refunded merchandise value (electronics return rates are high: 8–14%).
- **Marketing spend** — paid media across Paid Search, Paid Social, Display, Affiliate, plus email tooling.
- **Shipping cost** — what VoltEdge pays carriers (distinct from shipping *fees* charged to customers).
- **Payment processing** — merchant fees per transaction + financing cost of merchant-funded installments (\$3).
- **Inventory purchases** — cash paid to suppliers for stock, on supplier payment terms (\$4).

Financial definitions used throughout

Term	Definition
Gross Revenue (GMV)	Σ (unit list price × quantity), before discounts, excl. tax & shipping
Discounts	Σ discount amount applied to order lines
Returns	Σ net merchandise value of returned lines (post-discount)
Net Revenue	Gross Revenue – Discounts – Returns. <i>This is the North Star.</i> Excludes tax and shipping fees.
COGS	Σ (unit cost × net quantity sold)
Gross Profit	Net Revenue – COGS
Payment Fees	Merchant discount rate × amount + fixed fee per transaction (+ merchant-funded financing cost)
Contribution Margin	Gross Profit – Marketing Spend – Shipping Cost – Payment Fees (operating-contribution proxy; not a full P&L)

Net Revenue, COGS and margin are always expressed in **USD (reporting currency)**. Transactions are captured in **local currency** and converted using the exchange rate on the transaction date (see §6).

3. Payment methods, fees & customer-side cash

A deliberate CFO-grade thread. Payment choice affects **cost** (processing fees differ a lot by scheme) and **cash timing** (settlement lag, and Brazilian installments spreading cash over months).

`dim_payment_method` attributes

Attribute	Meaning
<code>method_group</code>	Card · Digital Wallet · Bank Transfer · BNPL · Pix · Boleto
<code>method_name</code>	Visa Credit, Mastercard Debit, Amex, Elo Credit, Hipercard, PayPal, Apple Pay, Google Pay, Klarna, Clearpay, SEPA Direct Debit, Pix, Boleto Bancário
<code>card_scheme</code>	Visa · Mastercard · Amex · Elo · Hipercard · (n/a)
<code>mdr_pct</code>	Merchant discount rate (% of transaction)
<code>fixed_fee_local</code>	Flat fee per transaction
<code>settlement_days</code>	Days until VoltEdge receives the cash
<code>installment_eligible</code>	Supports paying in instalments
<code>max_installments</code>	Cap (12 in BR, 3–4 for EU BNPL)
<code>available_markets</code>	Methods are market-specific (Pix/Boleto/Elo/Hipercard = BR only; SEPA/Klarna/Clearpay = UK/DE; Amex skewed US/UK)

Indicative economics (modeled, not a real rate card)

Method	<code>mdr_pct</code>	Fixed fee	Settlement days	Notes
Visa / Mastercard – Debit	1.1 %	—	2	cheapest card
Visa / Mastercard – Credit	2.2 %	—	2	
Amex	3.4 %	—	3	highest scheme fee
Elo / Hipercard (BR) – Credit	2.6 %	—	2 (à vista)	instalments settle monthly
PayPal	2.9 %	0.30	1	
Apple Pay / Google Pay	2.0 %	—	2	rides card rails
Klarna / Clearpay (BNPL)	4.5 %	0.30	2	merchant paid upfront, keeps the fee
SEPA Direct Debit (DE)	0.8 %	0.25	3	
Pix (BR)	0.45 %	—	0	instant, cheapest overall
Boleto Bancário (BR)	—	3.49	1 (after customer pays, avg +2)	~12 % of boletos are never paid → order cancelled

Installments (`installment_plan`)

Plan	Who pays financing	Cash-flow effect
<code>None</code> (à vista)	—	Full amount settles once, after <code>settlement_days</code>
<code>Interest-Free</code> (merchant-funded — BR "parcelado sem juros")	VoltEdge absorbs ~1.8 %/month of the deferred balance	Cash arrives in <i>N</i> monthly tranches; a financing cost is booked
<code>With Interest</code> (customer-funded)	Customer	Cash arrives in <i>N</i> monthly tranches; no extra merchant cost

- `fact_orders` carries the order-grain payment attributes and total `payment_fee`.
- `fact_payment_schedule` explodes each order into **one row per settlement tranche** (`installment_number`, `due_date`, `gross_amount`, `fee_amount`, `financing_cost`, `net_amount`, currency) — the table the customer-side cash-flow visuals sit on.

4. Inventory, procurement & the cash conversion cycle

The other half of working capital: cash goes **out** to suppliers to buy stock, that stock **sits** in a warehouse, then a sale converts it back to cash **in**. The report treats this as a first-class CFO/COO topic.

What is modeled

Object	Grain	Purpose
dim_supplier	supplier	Country, payment_terms_days (Net 30 / 45 / 60), lead_time_days , on-time-supply reliability
fact_purchase_orders	PO line (PO × SKU)	Order date, supplier, qty ordered, unit cost, PO value (local + USD), expected & actual receipt date, supplier payment due date (receipt + terms), supplier paid date
fact_inventory_movement	movement	Transactional ledger: receipt (from a PO), sale (from an order line), return_to_stock , adjustment , transfer . Date, SKU, warehouse, qty (+/-), unit cost, value
fact_inventory_snapshot	month-end × SKU × warehouse	Derived from the movement ledger. Units on hand, units in transit (inbound), units reserved, unit cost, inventory value (local + USD), days of cover

Building the snapshot **from** the movement ledger is a deliberate data-engineering showcase (a running balance / window function per SKU-warehouse), documented in [04_etl_pipeline.md](#) .

COGS sourcing

COGS on a sale is taken from the **weighted-average purchase cost** of stock on hand for that SKU at sale time (moving-average cost), so margin reflects what the goods actually cost — not a static list cost. Price drift in fact_purchase_orders therefore flows through to margin.

Working-capital KPIs (Executive Summary tile + dedicated Phase-2 section)

KPI	Definition	Benchmark
Inventory Value (on hand)	Σ units on hand × moving-avg cost, USD	—
Inventory in Transit	Σ inbound units not yet received × PO cost	—
DIO — Days Inventory Outstanding	Avg Inventory Value ÷ COGS × days in period	45–75
DSO — Days Sales Outstanding	Amount-weighted avg days from order date to cash settlement	US/EU ~2–4; BR much higher (instalments)
DPO — Days Payables Outstanding	Avg Accounts Payable ÷ COGS (or purchases) × days in period	30–60
CCC — Cash Conversion Cycle	DIO + DSO – DPO	lower is better; watch the trend
Accounts Payable (open)	Σ received-but-unpaid PO value at a date	—
Accounts Receivable (open)	Σ booked-but-unsettled customer amounts at a date	—
Working Capital Tied Up	Inventory Value + AR – AP	—
Supplier Spend	Σ PO value received in period, USD	—
Inventory Turnover	Trailing COGS ÷ Avg Inventory Value	5–8×/yr
Weeks of Cover	Inventory on hand ÷ avg weekly units sold	4–10
Stockout Rate %	SKU-days out of stock ÷ SKU-days	< 3%
Aged / Excess Inventory %	Value with > 90 days of cover ÷ total inventory value	< 15%

Company cash-flow timeline

A single view combining, per week/month:

- **Cash out** — supplier payments (PO value on supplier_paid_date), marketing spend, shipping cost, payment processing fees.
- **Cash in** — customer settlements from fact_payment_schedule .
- **Net cash flow** and **cumulative cash position**, plus the split of working capital between inventory and receivables. Makes the cost of BR "parcelado sem juros" and of slow-moving stock visible in cash terms.

5. Total vs Comparable Base (like-for-like)

Because markets go live mid-window, a raw YoY number mixes *organic* growth with *expansion* growth. Every headline growth metric is shown **two ways**:

View	Definition
Total	All markets, as reported.
Comparable Base	Only markets live for the entire CY + PY span → United States only. The like-for-like / "same-store" view.
Expansion Contribution	Total – Comparable Base. Revenue / orders / customers attributable to UK, DE, BR.

Secondary cut: **Core-3 Comparable** (US + UK + DE, comparable from 2025-01) for analysis from 2025 onward.

Implementation: `dim_market` carries `live_date` ; a calculated `is_comparable_base` (relative to PY start 2024-07-01) flags US. DAX also provides a **dynamic** comparable-base measure that recomputes the eligible market set from the visual's date context. Both are in [06_dax_measures.md](#) .

6. Multi-currency handling

- Every monetary transaction is stored in its **local currency** with a `currency_code` .
- `fact_exchange_rate` holds a **daily** rate per currency → USD.
- Curated fact tables carry **both** `amount_local` and a materialized `amount_usd` (converted at transaction-date rate) so results are reproducible without the model.
- DAX also exposes **currency-aware measures** and a **reporting-currency selector** (USD / EUR / GBP / BRL). Default = USD.
- FX approach: rates fluctuate around anchors (EUR 1.08, GBP 1.26, BRL 0.19 USD per unit) with realistic daily drift.

7. Seasonality & patterns embedded in the data

- **Q4 peak** — Nov/Dec ~1.7–2.1× a baseline month; **Black Friday / Cyber Monday** spike within November.
- **Secondary peaks** — mid-year sale (July), back-to-school (late Aug / Sep).
- **Growth trend** — ~2–4% MoM growth on the comparable base, plus step changes at each market launch.
- **Channel mix evolution** — heavy Paid Search/Social early; Organic, Email, Direct grow as brand awareness builds; CAC rises over time.
- **Category trends** — Wearables and Smart Home grow share; Smartphones' share slowly declines.
- **Purchase-cost drift** — supplier unit costs move over time; some SKUs discontinued and replaced → margin and inventory-aging effects.
- **Promo periods** — recurring discount events with elevated discount rates and order volume.
- **Payment-mix shift** — Pix share in BR grows MoM; BNPL share in UK/DE grows; Amex share slowly falls.
- **Fulfillment stress** — delivery times and late-delivery rates worsen during Q4 and vary by carrier and destination region.
- **Returns** — higher for Smartphones/Laptops, lower for Accessories; split "defective" vs "changed mind" / "not as described".

8. Stakeholders & the questions they ask

The report is **executive-level**: each page answers the standing questions of one leader.

Persona	Owns	Standing questions
CEO	Whole business	Are we hitting plan? Which market/category drives or drags growth? Revenue <i>and</i> margin trajectory? Total vs like-for-like?
CFO	Margin, cash & efficiency	Gross margin trend and why it moved. Discount leakage. Payment fees and the cost of "sem juros". Cash received vs revenue booked. Cash conversion cycle — how much cash is tied up in inventory and receivables? Contribution margin by market. Revenue vs target.
CMO	Demand & acquisition	CAC and ROAS by channel and campaign. New vs returning revenue. Which campaigns paid back? Funnel drop-off.
VP E-commerce	Site & conversion	Traffic, conversion rate, AOV, cart abandonment, revenue per session, device performance.
COO / VP Operations	Fulfillment & inventory	On-time delivery, delivery cycle time, shipping cost per order, carrier performance, stockouts and weeks of cover, aged inventory , inventory turnover, supplier lead time & reliability.
VP Customer	Retention & service	Repeat rate, cohort retention, CLV, churn, support volume & resolution time, CSAT, return rate by reason.

9. KPI framework

Metrics defined in business terms here; exact DAX in [06_dax_measures.md](#) . "Benchmark" is a plausible DTC-electronics reference range used to set report targets — not a claim about any real company.

9.0 North Star

KPI	Definition	Benchmark
Net Revenue (USD)	Gross Revenue – Discounts – Returns, converted to USD	vs monthly plan

9.1 Sales & Growth (Executive Summary, Sales Performance)

KPI	Definition	Benchmark
Gross Revenue (GMV)	$\Sigma (\text{list price} \times \text{qty})$	—
Net Revenue	Gross Revenue – Discounts – Returns	vs target
Orders	Distinct orders, excluding fully cancelled	—

KPI	Definition	Benchmark
Units Sold	Σ net quantity	—
Average Order Value (AOV)	$\text{Net Revenue} \div \text{Orders}$	\$170–\$260
Units per Order	$\text{Units} \div \text{Orders}$	1.6–2.2
Gross Profit	$\text{Net Revenue} - \text{COGS}$	—
Gross Margin %	$\text{Gross Profit} \div \text{Net Revenue}$	12–20% blended (thin, by design — see note below)
Discount Rate %	$\text{Discounts} \div \text{Gross Revenue}$	4–10%
Return Rate %	$\text{Returned Units} \div \text{Units Sold}$	7–12%
Warranty Attach Rate %	$\text{Orders with a warranty line} \div \text{eligible orders}$	6–12%
Net Revenue YoY % — Total	$(\text{CY} - \text{PY}) \div \text{PY}$, all markets	—
Net Revenue YoY % — Comparable Base	same, US-only like-for-like	—
Expansion Contribution	$\text{Total Net Revenue} - \text{Comparable-Base Net Revenue}$	—
Revenue vs Target %	$\text{Net Revenue} \div \text{Target} - 1$	≥ 0
Price / Volume / Mix	Decomposition of Net Revenue Δ vs PY into rate, quantity, category-mix effects	—
Revenue by Market / Category / Channel	Net Revenue sliced by dimension	—

Thin margin is the story, not a bug. Blended gross margin sits at ~14% — flagship phones, laptops and consoles carry almost no margin; audio and accessories carry it all. After marketing (~\$4M), shipping and payment fees, **contribution margin is near break-even** — VoltEdge is a fast-growing, not-yet-profitable business. The report is built to make that tension legible: *are we buying growth we can't afford, and does category mix / repeat revenue get us to profit?*

9.2 Payments & Customer Cash (*Executive Summary, Sales Performance > Payments*)

Payment Processing Cost · Effective Fee Rate % · Fee Rate by Scheme/Method/Market · Merchant-Funded Financing Cost · Installment Mix % · Avg Instalments (BR) · Revenue Booked vs Cash Received · Accounts Receivable (open) · DSO. (Definitions in §3.)

9.3 Working Capital & Inventory (*Executive Summary tile; dedicated Phase-2 section*)

DIO · DSO · DPO · **CCC** · Inventory Value · Inventory in Transit · Accounts Payable (open) · Accounts Receivable (open) · Working Capital Tied Up · Supplier Spend · Inventory Turnover · Weeks of Cover · Stockout Rate % · Aged / Excess Inventory % · Company cash-flow timeline. (Definitions in §4.)

9.4 Marketing & Acquisition (*Marketing & Acquisition*)

KPI	Definition	Benchmark
Marketing Spend	Σ paid media spend (+ email tooling)	—
Impressions / Clicks	From ad-platform exports	—
CTR %	$\text{Clicks} \div \text{Impressions}$	0.7–2.5%
CPC	$\text{Spend} \div \text{Clicks}$	\$0.40–\$1.60
New Customers	Customers whose first-ever order falls in the period	—
CAC (paid)	$\text{Paid Marketing Spend} \div \text{New Customers via paid channels}$	\$45–\$110
Blended CAC	$\text{Total Marketing Spend} \div \text{Total New Customers}$	\$30–\$70
ROAS	$\text{Attributed Net Revenue} \div \text{Ad Spend (last-touch, channel-level)}$	3.0–6.0×
MER	$\text{Total Net Revenue} \div \text{Total Marketing Spend}$	6–10×
New vs Returning Revenue	Net Revenue split by customer order sequence	—
Campaign Performance	Spend, Orders, Net Revenue, ROAS per campaign	ROAS $\geq 3\times$ to pay back
Acquisition Funnel	Impressions → Clicks → Sessions → Add-to-Cart → Orders	—

9.5 Website / Digital (*Phase 2*)

Sessions / Users · Conversion Rate % (Orders $\div$ Sessions, 1.3–2.6%) · Bounce Rate % · Pages per Session · Add-to-Cart Rate % · Cart Abandonment Rate % · Revenue per Session · Device / Browser mix.

9.6 Logistics & Fulfillment (Phase 2)

Avg Delivery Time (days, 3–6) · On-Time Delivery % (92–96%) · Perfect Order Rate % (88–93%) · Ship Time (days) · Shipping Cost per Order (\$6–\$14) · Shipping Cost Recovery % · Carrier Scorecard · Supplier Lead Time & On-Time-Supply % · (inventory metrics in §9.3).

9.7 CRM / Customer (Phase 3)

Active Customers · New / Returning · Repeat Purchase Rate % (25–35%) · Cohort Retention % · Churn Rate % · CLV · RFM Segment · Loyalty Tier Mix · Support Tickets · Avg Resolution Time (h, <24) · First Contact Resolution % (70–80%) · CSAT (≥4.3) · Return Rate by Reason.

10. Targets / plan

`fact_target` holds a **monthly plan by market** for: Net Revenue, Orders, New Customers, Blended CAC, Gross Margin %. Targets are "ambitious but plausible" (actuals land ~90–110% of plan). Targets exist only for months a market is live.

11. Out of scope (explicit)

- Full P&L / GAAP accounting, tax filing detail, statutory cash-flow statement.
- Real ad-platform attribution modeling (simple last-touch at channel level).
- Employee, payroll, or fixed-cost data.
- Real personal data — all customers synthetic (`Faker` , seeded).
- Real-time / streaming — batch pipeline, one historical load.

12. Resolved assumptions

#	Assumption	Decision
1	North Star = Net Revenue (excl. tax & shipping fees), USD; Gross Profit secondary	✓
2	Return rate ~8%, blended gross margin ~14% (thin by design; growth-stage, near break-even after marketing)	✓
3	Extract window 24 months : 2024-07-01 → 2026-06-30	✓
4	Expansion inside window: US always-on · UK+DE 2025-01-01 · BR 2025-07-01	✓
5	Every growth metric shown Total and Comparable Base (US like-for-like) + Expansion Contribution	✓
6	Warranty / protection-plan revenue modeled as an add-on line	✓
7	Reporting-currency selector (USD/EUR/GBP/BRL) in the model	✓
8	Payment model: MDR by scheme, fixed fees, settlement lag, instalments (merchant- vs customer-funded), <code>fact_payment_schedule</code>	✓
9	Procurement & inventory modeled: <code>dim_supplier</code> (Net 30/45/60), <code>fact_purchase_orders</code> , <code>fact_inventory_movement</code> ledger → derived <code>fact_inventory_snapshot</code> ; moving-average COGS; DIO/DSO/DPO/CCC and a company cash-flow timeline	✓
10	Realised scale: 294 SKUs · ~103k customers · ~158k non-cancelled orders · ~262k order lines · ~14k PO lines · 24 months. AOV ~\$208, ~1.7 lifetime orders/customer, repeat rate ~40%.	✓

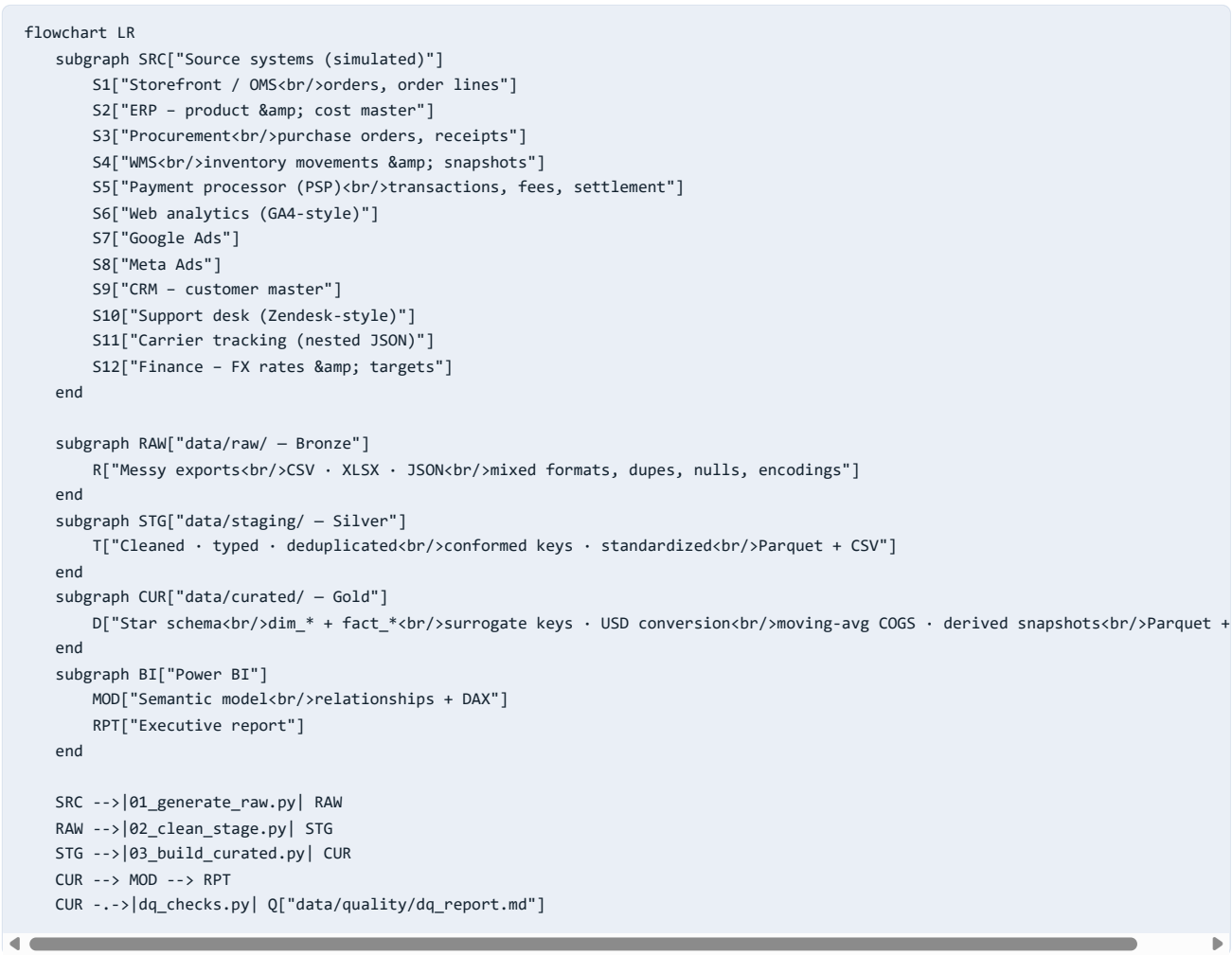
02 — Data Architecture

How raw, messy, multi-source data becomes a clean, documented star schema that Power BI consumes. The design follows a **medallion architecture** (raw → staging → curated) with a seeded, reproducible Python pipeline.

1. Design principles

Principle	How it's applied
Reproducible	Single random seed (<code>etl/config.py</code>). Re-running <code>run_pipeline.py</code> reproduces byte-identical data.
Layered	Each layer has one job. Raw is never edited; staging is deterministic from raw; curated is deterministic from staging.
Idempotent	Every script fully rewrites its outputs. No incremental state, no partial runs.
Source-faithful	Raw files imitate real system exports — different schemas, formats, encodings, and the data-quality problems those systems actually produce.
Documented	Every table and column is described in 03 data dictionary.md . Every transformation in 04 etl pipeline.md .
Tested	<code>dq_checks.py</code> runs assertions (row counts, keys, referential integrity, null rules, value ranges, accounting identities) and fails the build on violation.
BI-ready	Curated layer is a Kimball-style star schema: conformed dimensions, additive facts, surrogate keys, a real date dimension.

2. Layer overview



Bronze — `data/raw/`

Generated by `01_generate_raw.py` . Files look like exports pulled by an analyst on their first day: no two systems agree on formats, and each carries its own defects (§4). **Nothing downstream ever writes here.**

Silver — `data/staging/`

Produced by `02_clean_stage.py` . One staging table per meaningful entity. Work done: parsing and typing (dates, money, booleans), trimming/casing, de-duplication, currency normalization, category/brand standardization, unioning the two ad platforms, flattening the carrier JSON, resolving customer identity, and reconciling PSP transactions to orders. Every row dropped or altered is logged to the DQ report.

Gold — `data/curated/`

Produced by `03_build_curated.py` . The dimensional model:

Dimensions

Table	Notes
<code>dim_date</code>	2024-07-01 → 2026-06-30 + buffer; fiscal cols, holiday flags (US/UK/DE/BR), promo-period flag
<code>dim_market</code>	US/UK/DE/BR: currency, <code>live_date</code> , <code>is_comparable_base</code>
<code>dim_customer</code>	Acquisition channel & date, geography, segment, loyalty tier, marketing consent
<code>dim_geography</code>	Country → region → state/city (per-market hierarchy)
<code>dim_product</code>	Category → Subcategory → Brand → Product; launch date, <code>is_active</code> , weight, warranty months
<code>dim_supplier</code>	Country, <code>payment_terms_days</code> (Net 30/45/60), <code>lead_time_days</code> , on-time-supply reliability
<code>dim_channel</code>	Organic Search, Paid Search, Paid Social, Display, Email, Direct, Affiliate, Referral
<code>dim_campaign</code>	Type, market, start/end, target segment
<code>dim_payment_method</code>	<code>method_group</code> , <code>card_scheme</code> , <code>mdr_pct</code> , <code>fixed_fee_local</code> , <code>settlement_days</code> , instalment attributes, <code>available_markets</code>
<code>dim_carrier</code>	Market, service level, baseline SLA days
<code>dim_currency</code>	USD/EUR/GBP/BRL; <code>is_reporting_currency</code>

Facts

Table	Grain	Key measures
<code>fact_order_lines</code>	order line (order × product × line_type)	qty, unit_price, discount, gross/net revenue, COGS (moving-avg), margin — <code>*_local</code> + <code>*_usd</code>
<code>fact_orders</code>	order header	order total, <code>payment_method_key</code> , <code>installment_count</code> , <code>installment_plan</code> , <code>payment_fee</code> , tax, shipping fee/cost, promised & delivered dates, status
<code>fact_payment_schedule</code>	order × settlement tranche	<code>installment_number</code> , <code>due_date</code> , gross / fee / financing_cost / net amount
<code>fact_purchase_orders</code>	PO × SKU	qty ordered, unit cost, PO value, expected/actual receipt date, supplier payment due & paid dates
<code>fact_inventory_movement</code>	movement	type (receipt/sale/return_to_stock/adjustment/transfer), qty ±, unit cost, value
<code>fact_inventory_snapshot</code>	month-end × SKU × warehouse	units on hand / in transit / reserved, unit cost, inventory value, days of cover — derived from the movement ledger
<code>fact_web_traffic_daily</code>	date × market × channel × device	sessions, users, pageviews, bounces, add-to-carts, carts, orders-attributed
<code>fact_marketing_spend</code>	date × market × channel × campaign	spend, impressions, clicks — <code>*_local</code> + <code>*_usd</code>
<code>fact_returns</code>	return line	returned qty, refund value, reason code, condition
<code>fact_support_tickets</code>	ticket	created/resolved timestamps, category, priority, first-contact-resolution flag, CSAT
<code>fact_exchange_rate</code>	date × currency	rate to USD
<code>fact_target</code>	month × market × metric	target value

- Surrogate integer keys on every dimension; facts carry keys + measures + a few degenerate dimensions (`order_id` , `po_number` , `ticket_id`).
- Money columns stored as `*_local` **and** `*_usd` (converted at transaction-date rate).
- Both Parquet (Power BI / analytical use) and CSV (portability, review) written for every curated table.

3. Source system inventory

#	Simulated system	Represents	Raw file(s)	Format
1	Storefront / OMS	Orders, order lines, customer-as-seen-at-checkout	<code>oms_orders.csv</code> , <code>oms_order_lines.csv</code>	CSV
2	ERP	Product master, unit-cost history, supplier links	<code>erp_products.xlsx</code>	XLSX (multi-sheet)

#	Simulated system	Represents	Raw file(s)	Format
3	Procurement	Purchase orders, receipts, supplier terms	procurement_purchase_orders.csv , suppliers.csv	CSV
4	WMS	Inventory movements + month-end snapshots	wms_inventory_movements.csv , wms_inventory_snapshots.csv	CSV
5	Payment processor (PSP)	Transaction-level auth, fees, settlement dates, instalment plans	psp_transactions.csv	CSV
6	Web analytics	Daily sessions / engagement / funnel by market-channel-device	web_analytics_daily.csv	CSV
7	Google Ads	Daily paid-search spend, impressions, clicks, campaign	google_ads_report.csv	CSV
8	Meta Ads	Daily paid-social spend, impressions, clicks (different column names/currency)	meta_ads_export.csv	CSV
9	CRM	Customer master: acquisition, geography, segment, consent	crm_customers.csv	CSV
10	Support desk	Support tickets: category, timestamps, CSAT	support_tickets.csv	CSV
11	Carrier tracking	Shipment + tracking events per order	carrier_tracking.json	JSON (nested)
12	Finance	Daily FX rates to USD; monthly plan/targets by market	fx_rates.csv , finance_targets.csv	CSV

4. Data-quality issues deliberately injected

Handled by `02_clean_stage.py` ; every occurrence is counted in the DQ report.

Area	Injected problem	Cleaning response
Dates	Mixed formats (<code>2025-03-05</code> , <code>03/05/2025</code> , <code>5-Mar-2025</code>), some Excel serials	Multi-format parse → ISO date
Money	<code>"\$1,299.00"</code> , <code>"1.299,00"</code> (EU), <code>"R\$ 1.299,00"</code> , blanks	Locale-aware parse → decimal
Customers	Duplicate rows, <code>EMAIL</code> vs <code>email</code> , trailing spaces, mixed casing, ~2% null email	Dedup on natural key, normalize, flag missing email
Products	Category spelled 3 ways (<code>Smartphones</code> / <code>smart phones</code> / <code>Smart-Phones</code>), brand casing	Mapping table → canonical category / brand
Ad platforms	Google vs Meta: different column names, currencies, date grains	Rename to common schema, convert to USD, align to daily
PSP ↔ OMS	~1% of PSP transactions don't match an order id; a few orders have no PSP row	Reconciliation report; unmatched flagged, not force-joined
Web analytics	~1.5% of dates missing; occasional negative / double-counted sessions	Gap report, clamp/flag anomalies
Orders	~0.5% duplicated order lines; a few order ids with no customer; negative quantities	Dedup, orphan report, route negatives to returns
Carrier JSON	Nested events array; status vocab <code>delivered</code> / <code>DELIVERED</code> / <code>Delivery complete</code>	Flatten, status-vocabulary map
Encoding	UTF-8 vs Latin-1 in Brazil rows (<code>SÃ£o Paulo</code>)	Encoding detection + fix
Referential	Some SKUs in orders not in product master (discontinued)	Late-arriving-dimension stub + report
Inventory	Movement ledger vs snapshot disagree for a few SKU-warehouses	Reconcile to ledger; variance reported

5. Tech stack

Tool	Use
Python 3.12	Pipeline language
pandas	Transformations
pyarrow	Parquet I/O
NumPy	Seeded random generation, statistical patterns
Faker	Synthetic names, addresses, emails (seeded)
openpyxl	Read/write the ERP <code>.xlsx</code>
python-dateutil	Multi-format date parsing
DuckDB (optional)	Ad-hoc SQL validation of curated layer

Tool	Use
Power BI Desktop	Semantic model + report
pbir CLI	Version-controlled Power BI project (.pbip / PBIR)

etl/requirements.txt pins exact versions.

6. Running & refreshing

```
cd etl
python -m pip install -r requirements.txt
python run_pipeline.py      # 01_generate_raw → 02_clean_stage → 03_build_curated → dq_checks
```

Step	Script	Reads	Writes
1	01_generate_raw.py	config.py	data/raw/*
2	02_clean_stage.py	data/raw/*	data/staging/*, DQ log fragments
3	03_build_curated.py	data/staging/*	data/curated/*
4	dq_checks.py	data/curated/*	data/quality/dq_report.md (exit ≠ 0 on failure)

Power BI connects to data/curated/*.parquet via a parameterized folder path (pDataFolder), so the report is portable: change one parameter to point at the curated folder on any machine.

7. Naming conventions

Object	Convention	Example
Raw files	<system>_<entity>.<ext> , snake_case	google_ads_report.csv
Staging tables	stg_<entity>	stg_order_lines
Curated dims	dim_<entity> (singular)	dim_customer
Curated facts	fact_<process>	fact_order_lines
Surrogate keys	<entity>_key (int)	customer_key
Natural/business keys	<entity>_id / <entity>_number (from source)	order_id , po_number
Money columns	<measure>_local / <measure>_usd	net_revenue_usd
Dates	ISO date ; keys as yyyyymmdd int	order_date , order_date_key
Booleans	is_<state>	is_returned
DAX measures	Title Case with spaces, in measure tables	Net Revenue (USD)

03 — Data Dictionary

Generated by `etl/gen_data_dictionary.py` from the files on disk.

Three layers: **raw** (source-faithful, messy) → **staging** (cleaned & typed) → **curated** (star schema, BI-ready). Row counts and null rates are from the current seeded run.

Raw layer — data/raw/

fx_rates.csv

Finance — daily USD-per-unit FX rates (EU date format, USD row omitted).

2,310 rows

column	dtype	null %	sample
date	str	0.0	22/05/2024
ccy	str	0.0	EUR
usd_per_unit	str	0.0	1.120146

suppliers.csv

Procurement — supplier master: country, payment terms, lead time, reliability.

15 rows

column	dtype	null %	sample
supplier_id	str	0.0	SUP-001
supplier_name	str	0.0	Shenzhen Components
country	str	0.0	KR
payment_terms_days	str	0.0	30
lead_time_days	str	0.0	21
on_time_rate	str	0.0	0.905

erp_products.xlsx

ERP — product master + monthly cost_history + price_history (3 sheets); category spellings intentionally inconsistent.

sheet **products** — 294 rows

column	dtype	null %	sample
SKU	str	0.0	SKU-00001
Description	str	0.0	Rivon Mid-range Smartphones 656
category	str	0.0	SMARTPHONES
subcategory	str	0.0	Mid-range
brand	str	0.0	rivon
list_price_usd	str	0.0	\$ 399.00
unit_cost_usd	float64	0.0	378.49
launch_date	str	0.0	05/22/2025
discontinue_date	str	83.7	01/29/2026
is_warranty_eligible	bool	0.0	True
weight_kg	float64	0.0	0.519
warranty_months	int64	0.0	12
supplier_id	str	0.0	SUP-014

sheet **cost_history** — 9,033 rows

column	dtype	null %	sample
sku_id	str	0.0	SKU-00001
effective_date	str	0.0	2025-05-01
unit_cost_usd	float64	0.0	376.54

sheet **price_history** — 9,033 rows

column	dtype	null %	sample
sku_id	str	0.0	SKU-00001
effective_date	str	0.0	2025-05-01
list_price_usd	float64	0.0	399.0

crm_customers.csv

CRM — customer master (utf-8 region files).

91,313 rows

column	dtype	null %	sample
customer_id	str	0.0	CUST-002289
market	str	0.0	US
signup_date	str	0.0	2023-12-23
acquisition_channel	str	0.0	Paid Social
full_name	str	0.0	Shawn Rose
email	str	1.8	shawn.rose819@proton.me
city	str	0.0	Brownland
state_region	str	0.0	KS
postal_code	str	0.0	75421
segment	str	0.0	Prosumer
birth_year	str	0.0	1975
marketing_consent	str	0.0	False

crm_customers_br.csv

CRM — Brazil customer master, exported in Latin-1.

12,996 rows

column	dtype	null %	sample
customer_id	str	0.0	CUST-079600
market	str	0.0	BR
signup_date	str	0.0	2026-01-03
acquisition_channel	str	0.0	Paid Search
full_name	str	0.0	Emilly Castro
email	str	1.7	antony.teixeira623@gmail.com
city	str	0.0	Macedo
state_region	str	0.0	AL
postal_code	str	0.0	83261358
segment	str	0.0	Consumer
birth_year	str	0.0	2001
marketing_consent	str	0.0	False

oms_orders.csv

Storefront/OMS — order headers; mixed date & money formats.

159,405 rows

column	dtype	null %	sample
order_id	str	0.0	US-202407-0000001
customer_id	str	0.4	CUST-007134
market	str	0.0	US
order_ts	str	0.0	2024-07-01 00:16:00
currency	str	0.0	USD
channel	str	0.0	Paid Social
device	str	0.0	Mobile
warehouse	str	0.0	FC-DAL
carrier	str	0.0	UPS
ship_date	str	0.0	2024-07-04
promised_date	str	0.0	2024-07-10
delivered_date	str	0.7	2024-07-05
is_late	str	0.0	False
order_status	str	0.0	delivered
units	str	0.0	2
gross_amount_local	str	0.0	214.98
discount_amount_local	str	0.0	16.75
net_amount_local	str	0.0	198.23
tax_amount_local	str	0.0	13.88
shipping_fee_local	str	0.0	0.00
shipping_cost_usd	str	0.0	13.36
payment_method	str	0.0	Mastercard Credit
card_scheme	str	22.3	Mastercard
installments	str	0.0	1
installment_plan	str	100.0	
payment_fee_local	str	0.0	4.67

oms_order_lines.csv

Storefront/OMS — order lines; a few duplicates & negative quantities.

263,376 rows

column	dtype	null %	sample
order_line_id	str	0.0	OL-00153335
order_id	str	0.0	BR-202511-0106878
sku_id	str	0.0	SKU-00160
line_type	str	0.0	product
quantity	str	0.0	1
unit_price_local	str	0.0	160.99
discount_amount_local	str	0.0	0.00
gross_amount_local	str	0.0	160.99
net_amount_local	str	0.0	160.99
currency	str	0.0	BRL

psp_transactions.csv

Payment processor — one auth per order (+ unmatched noise); fee column mislabelled.

160,332 rows

column	dtype	null %	sample
psp_txn_id	str	0.0	TXN2024070100110454
order_ref	str	0.0	US-202601-0124747
method	str	0.0	Visa Credit
scheme	str	30.8	Visa
amount	str	0.0	480.43
ccy	str	0.0	USD
mdr_pct	str	0.0	10.57
installments	str	0.0	1
plan	str	92.1	Interest-Free
auth_ts	str	0.0	2026-01-20 20:41:00
status	str	0.0	authorized

procurement_purchase_orders.csv

Procurement — PO lines with receipt, due and paid dates.

13,732 rows

column	dtype	null %	sample
po_number	str	0.0	PO-000001
supplier_id	str	0.0	SUP-015
sku_id	str	0.0	SKU-00035
destination_warehouse	str	0.0	FC-DAL
market	str	0.0	US
order_date	str	0.0	2024-06-11
qty_ordered	str	0.0	50
unit_cost_usd	str	0.0	177.57
po_value_usd	str	0.0	8878.5
expected_receipt_date	str	0.0	2024-06-27
actual_receipt_date	str	0.0	2024-06-27
payment_terms_days	str	0.0	30
payment_due_date	str	0.0	2024-07-27
supplier_paid_date	str	0.0	2024-07-25

wms_inventory_movements.csv

WMS — stock movement ledger (adjustment / receipt / sale).

268,159 rows

column	dtype	null %	sample
movement_id	str	0.0	MOV-000000001
movement_date	str	0.0	2025-04-29
sku_id	str	0.0	SKU-00001
warehouse	str	0.0	FC-BHX
market	str	0.0	UK
movement_type	str	0.0	receipt
quantity	str	0.0	10

wms_inventory_snapshots.csv

WMS — month-end snapshots; ~3% perturbed vs the ledger.

26,532 rows

column	dtype	null %	sample
snapshot_date	str	0.0	2024-07-31
sku_id	str	0.0	SKU-00002
warehouse	str	0.0	FC-DAL
market	str	0.0	US
units_on_hand	str	0.0	105

returns.csv

Storefront — return lines with reason and restock flag.

20,375 rows

column	dtype	null %	sample
return_id	str	0.0	RET-0000001
order_id	str	0.0	US-202407-0000020
order_line_id	str	0.0	OL-00000019
sku_id	str	0.0	SKU-00154
return_date	str	0.0	27/07/2024
quantity	str	0.0	1
refund_amount_local	str	0.0	36.99
currency	str	0.0	USD
reason	str	0.0	Changed my mind
restocked	str	0.0	True
condition	str	0.0	sellable

web_analytics_daily.csv

Web analytics — daily sessions & funnel by market:channel-device.

34,950 rows

column	dtype	null %	sample
date	str	0.0	2024-07-01
market	str	0.0	US
channel	str	0.0	Affiliate
device	str	0.0	Desktop
sessions	str	0.0	71
users	str	0.0	61
pageviews	str	0.0	344
bounces	str	0.0	33
add_to_carts	str	0.0	6
carts_created	str	0.0	3
transactions	str	0.0	2

google_ads_report.csv

Google Ads — daily paid search + display.

4,374 rows

column	dtype	null %	sample
date	str	0.0	2024-07-01

column	dtype	null %	sample
market	str	0.0	US
campaign	str	0.0	US_PaidSearch_Q32024_Always-On
impressions	str	0.0	18632
clicks	str	0.0	993
campaign_type	str	0.0	Search
cost	str	0.0	821.33
currency	str	0.0	USD

meta_ads_export.csv

Meta Ads — daily paid social (different column names & date format).

2,187 rows

column	dtype	null %	sample
day	str	0.0	01/07/2024
country	str	0.0	US
ad_campaign	str	0.0	US_PaidSocial_Q32024_Category-Mobile
reach	str	0.0	24725
impressions	str	0.0	35322
link_clicks	str	0.0	337
spend	str	0.0	337.3
currency	str	0.0	USD

other_marketing_spend.csv

Finance — monthly affiliate commission + email tooling.

144 rows

column	dtype	null %	sample
month	str	0.0	2024-07
market	str	0.0	US
channel	str	0.0	Affiliate
cost_type	str	0.0	commission
amount_local	str	0.0	5498.08
currency	str	0.0	USD

support_tickets.csv

Support desk — tickets with timestamps, FCR flag and CSAT.

22,257 rows

column	dtype	null %	sample
ticket_id	str	0.0	TCK-0000001
order_id	str	0.0	US-202407-0000003
customer_id	str	0.0	CUST-006653
market	str	0.0	US
created_ts	str	0.0	2024-07-08 13:16:00
resolved_ts	str	0.0	2024-07-08 20:06:48
category	str	0.0	Return / refund
priority	str	0.0	Medium
first_contact_resolution	str	0.0	False

column	dtype	null %	sample
csat	str	0.0	5

carrier_tracking.json

Carrier — nested shipment + tracking events (sample); inconsistent status vocab.

14,287 records (nested `events` array flattened on load)

column	dtype	null %	sample
order_id	str	0.0	US-202407-0000030
carrier	str	0.0	USPS
tracking_number	str	0.0	USP9901674199
service_level	str	0.0	express
events	object	0.0	[{'status': 'Created', 'ts': '2024-07-03...

finance_targets.csv

Finance — monthly plan per market for 5 metrics.

360 rows

column	dtype	null %	sample
month	str	0.0	2025-07
market	str	0.0	BR
metric	str	0.0	Net Revenue
target_value	str	0.0	140772.51

Staging layer — data/staging/

Typed Parquet, one table per entity. Cleaning rules in [04_etl_pipeline.md](#).

stg_carrier_events — 57,148 rows

column	dtype	null %	sample
order_id	str	0.0	US-202407-0000030
carrier	str	0.0	USPS
status	str	0.0	created
event_ts	datetime64[us, UTC]	0.0	2024-07-03 00:00:00+00:00
location	str	0.0	US

stg_cost_history — 9,033 rows

column	dtype	null %	sample
sku_id	str	0.0	SKU-00001
effective_date	datetime64[us]	0.0	2025-05-01 00:00:00
unit_cost_usd	float64	0.0	376.54

stg_customers — 102,767 rows

column	dtype	null %	sample
customer_id	str	0.0	CUST-002289
market	str	0.0	US
signup_date	datetime64[us]	0.0	2023-12-23 00:00:00
acquisition_channel	str	0.0	Paid Social
full_name	str	0.0	Shawn Rose

column	dtype	null %	sample
email	str	0.0	shawn.rose819@proton.me
city	str	0.0	Brownland
state_region	str	0.0	KS
postal_code	str	0.0	75421
segment	str	0.0	Prosumer
birth_year	int64	0.0	1975
marketing_consent	bool	0.0	False
has_email	bool	0.0	True
acquired_before_window	bool	0.0	True

stg_fx_rates — 4,384 rows

column	dtype	null %	sample
date	datetime64[ms]	0.0	2024-01-01 00:00:00
currency_code	str	0.0	USD
rate_to_usd	float64	0.0	1.0

stg_inventory_movements — 268,159 rows

column	dtype	null %	sample
movement_id	str	0.0	MOV-000000001
movement_date	datetime64[us]	0.0	2025-04-29 00:00:00
sku_id	str	0.0	SKU-00001
warehouse	str	0.0	FC-BHX
market	str	0.0	UK
movement_type	str	0.0	receipt
quantity	int64	0.0	10

stg_inventory_snapshots — 26,532 rows

column	dtype	null %	sample
snapshot_date	datetime64[us]	0.0	2024-07-31 00:00:00
sku_id	str	0.0	SKU-00002
warehouse	str	0.0	FC-DAL
market	str	0.0	US
units_on_hand	int64	0.0	105
ledger_qty	int64	0.0	105
variance	int64	0.0	0

stg_marketing_spend — 10,935 rows

column	dtype	null %	sample
date	datetime64[us]	0.0	2024-07-01 00:00:00
market	str	0.0	US
channel	str	0.0	Paid Search
campaign	str	0.0	US_PaidSearch_Q32024_Always-On
impressions	int64	0.0	18632
clicks	int64	0.0	993
cost_local	float64	0.0	821.33

column	dtype	null %	sample
currency	str	0.0	USD
source_system	str	0.0	google_ads

stg_order_line_rejects — 480 rows

column	dtype	null %	sample
order_line_id	str	0.0	OL-00117320
order_id	str	0.0	US-202510-0084021
sku_id	str	0.0	SKU-00102
line_type	str	0.0	product
quantity	int64	0.0	-1
unit_price_local	float64	0.0	83.99
discount_amount_local	float64	0.0	0.0
gross_amount_local	float64	0.0	83.99
net_amount_local	float64	0.0	83.99
currency	str	0.0	USD
reject_reason	str	0.0	non-positive quantity

stg_order_lines — 261,589 rows

column	dtype	null %	sample
order_line_id	str	0.0	OL-00153335
order_id	str	0.0	BR-202511-0106878
sku_id	str	0.0	SKU-00160
line_type	str	0.0	product
quantity	int64	0.0	1
unit_price_local	float64	0.0	160.99
discount_amount_local	float64	0.0	0.0
gross_amount_local	float64	0.0	160.99
net_amount_local	float64	0.0	160.99
currency	str	0.0	BRL
order_known	bool	0.0	True
sku_known	bool	0.0	True

stg_orders — 159,405 rows

column	dtype	null %	sample
order_id	str	0.0	US-202407-0000001
customer_id	str	0.0	CUST-007134
market	str	0.0	US
order_ts	datetime64[ns]	0.0	2024-07-01 00:16:00
currency	str	0.0	USD
channel	str	0.0	Paid Social
device	str	0.0	Mobile
warehouse	str	0.0	FC-DAL
carrier	str	0.0	UPS
ship_date	datetime64[us]	0.0	2024-07-04 00:00:00

column	dtype	null %	sample
promised_date	datetime64[us]	0.0	2024-07-10 00:00:00
delivered_date	datetime64[us]	0.7	2024-07-05 00:00:00
is_late	bool	0.0	False
order_status	str	0.0	delivered
units	int64	0.0	2
gross_amount_local	float64	0.0	214.98
discount_amount_local	float64	0.0	16.75
net_amount_local	float64	0.0	198.23
tax_amount_local	float64	0.0	13.88
shipping_fee_local	float64	0.0	0.0
shipping_cost_usd	float64	0.0	13.36
payment_method	str	0.0	Mastercard Credit
card_scheme	str	0.0	Mastercard
installments	int64	0.0	1
installment_plan	str	0.0	None
payment_fee_local	float64	0.0	4.67
order_date	datetime64[ns]	0.0	2024-07-01 00:00:00
has_customer_id	bool	0.0	True
customer_known	bool	0.0	True

stg_price_history — 9,033 rows

column	dtype	null %	sample
sku_id	str	0.0	SKU-00001
effective_date	datetime64[us]	0.0	2025-05-01 00:00:00
list_price_usd	float64	0.0	399.0

stg_products — 294 rows

column	dtype	null %	sample
sku_id	str	0.0	SKU-00001
product_name	str	0.0	Rivon Mid-range Smartphones 656
category	str	0.0	Smartphones
subcategory	str	0.0	Mid-range
brand	str	0.0	Rivon
list_price_usd	float64	0.0	399.0
unit_cost_usd	float64	0.0	378.49
launch_date	datetime64[us]	0.0	2025-05-22 00:00:00
discontinue_date	datetime64[us]	83.7	2026-01-29 00:00:00
is_warranty_eligible	bool	0.0	True
weight_kg	float64	0.0	0.519
warranty_months	int64	0.0	12
supplier_id	str	0.0	SUP-014

stg_psp_transactions — 160,332 rows

column	dtype	null %	sample
psp_txn_id	str	0.0	TXN2024070100110454
order_ref	str	0.0	US-202601-0124747
method	str	0.0	Visa Credit
scheme	str	0.0	Visa
currency	str	0.0	USD
reported_fee_local	float64	0.0	10.57
installments	int64	0.0	1
plan	str	0.0	None
auth_ts	datetime64[us]	0.0	2026-01-20 20:41:00
status	str	0.0	authorized
amount_local	float64	0.0	480.43
order_matched	bool	0.0	True

stg_purchase_orders — 13,732 rows

column	dtype	null %	sample
po_number	str	0.0	PO-000001
supplier_id	str	0.0	SUP-015
sku_id	str	0.0	SKU-00035
destination_warehouse	str	0.0	FC-DAL
market	str	0.0	US
order_date	datetime64[us]	0.0	2024-06-11 00:00:00
qty_ordered	int64	0.0	50
unit_cost_usd	float64	0.0	177.57
po_value_usd	float64	0.0	8878.5
expected_receipt_date	datetime64[us]	0.0	2024-06-27 00:00:00
actual_receipt_date	datetime64[us]	0.0	2024-06-27 00:00:00
payment_terms_days	int64	0.0	30
payment_due_date	datetime64[us]	0.0	2024-07-27 00:00:00
supplier_paid_date	datetime64[us]	0.0	2024-07-25 00:00:00
late_receipt_days	int64	0.0	0

stg_returns — 20,375 rows

column	dtype	null %	sample
return_id	str	0.0	RET-0000001
order_id	str	0.0	US-202407-0000020
order_line_id	str	0.0	OL-00000019
sku_id	str	0.0	SKU-00154
return_date	datetime64[us]	0.0	2024-07-27 00:00:00
quantity	int64	0.0	1
refund_amount_local	float64	0.0	36.99
currency	str	0.0	USD
reason	str	0.0	Changed my mind
restocked	bool	0.0	True
condition	str	0.0	sellable

column	dtype	null %	sample
line_known	bool	0.0	True

stg_shipments — 14,287 rows

column	dtype	null %	sample
order_id	str	0.0	US-202407-0000030
carrier	str	0.0	USPS
tracking_number	str	0.0	USP9901674199
service_level	str	0.0	express
created_ts	datetime64[us, UTC]	0.0	2024-07-03 00:00:00+00:00
in_transit_ts	datetime64[us, UTC]	0.0	2024-07-04 00:00:00+00:00
out_for_delivery_ts	datetime64[us, UTC]	0.0	2024-07-07 18:00:00+00:00
delivered_ts	datetime64[us, UTC]	0.0	2024-07-08 00:00:00+00:00
n_events	int64	0.0	4

stg_suppliers — 15 rows

column	dtype	null %	sample
supplier_id	str	0.0	SUP-001
supplier_name	str	0.0	Shenzhen Components
country	str	0.0	KR
payment_terms_days	int64	0.0	30
lead_time_days	int64	0.0	21
on_time_rate	float64	0.0	0.905

stg_support_tickets — 22,257 rows

column	dtype	null %	sample
ticket_id	str	0.0	TCK-0000001
order_id	str	0.0	US-202407-0000003
customer_id	str	0.0	CUST-006653
market	str	0.0	US
created_ts	datetime64[us]	0.0	2024-07-08 13:16:00
resolved_ts	datetime64[us]	0.0	2024-07-08 20:06:48
category	str	0.0	Return / refund
priority	str	0.0	Medium
first_contact_resolution	bool	0.0	False
csat	int64	0.0	5
resolution_hours	float64	0.0	6.846666666666667

stg_targets — 360 rows

column	dtype	null %	sample
month	str	0.0	2025-07
market	str	0.0	BR
metric	str	0.0	Net Revenue
target_value	float64	0.0	140772.51
month_date	datetime64[us]	0.0	2025-07-01 00:00:00

stg_web_traffic — 34,950 rows

column	dtype	null %	sample
date	datetime64[us]	0.0	2024-07-01 00:00:00
market	str	0.0	US
channel	str	0.0	Affiliate
device	str	0.0	Desktop
sessions	int64	0.0	71
users	int64	0.0	61
pageviews	int64	0.0	344
bounces	int64	0.0	33
add_to_carts	int64	0.0	6
carts_created	int64	0.0	3
transactions	int64	0.0	2
had_data_anomaly	bool	0.0	False

Curated layer — data/curated/

Star schema (Parquet + CSV). Grain and relationships in [05_data_model.md](#).

dim_date — 1,096 rows

Calendar dimension; fiscal year starts 1 July; promo / Black-Friday / YoY flags.

column	dtype	null %	sample
date	datetime64[ms]	0.0	2024-01-01 00:00:00
date_key	Int64	0.0	20240101
year	int32	0.0	2024
quarter	int32	0.0	1
quarter_name	str	0.0	2024-Q1
month	int32	0.0	1
month_name	str	0.0	Jan
month_year	str	0.0	2024-01
month_start_date	datetime64[ms]	0.0	2024-01-01 00:00:00
day_of_month	int32	0.0	1
day_of_week	int32	0.0	1
day_name	str	0.0	Mon
is_weekend	bool	0.0	False
iso_week	int64	0.0	1
day_of_year	int32	0.0	1
fiscal_year	int32	0.0	2024
fiscal_quarter	int32	0.0	3
fiscal_period	int32	0.0	7
is_promo_period	bool	0.0	False
promo_name	str	0.0	
is_black_friday_week	bool	0.0	False
in_extract_window	bool	0.0	False
yoy_period	str	0.0	

dim_market — 4 rows

The four markets, their currency, go-live date and comparable-base flag.

column	dtype	null %	sample
market_key	int64	0.0	1
market_id	str	0.0	US
market_name	str	0.0	United States
region	str	0.0	North America
currency_code	str	0.0	USD
live_date	datetime64[ms]	0.0	2024-07-01 00:00:00
is_comparable_base	bool	0.0	True

dim_currency — 4 rows

USD / EUR / GBP / BRL with the reporting-currency flag.

column	dtype	null %	sample
currency_key	int64	0.0	1
currency_code	str	0.0	USD
currency_name	str	0.0	US Dollar
currency_symbol	str	0.0	\$
is_reporting_currency	bool	0.0	True

dim_customer — 102,768 rows

Customer dimension enriched with lifetime orders / revenue / status / loyalty tier.

column	dtype	null %	sample
customer_key	int64	0.0	1
customer_id	str	0.0	CUST-002289
market_id	str	0.0	US
signup_date	datetime64[us]	0.0	2023-12-23 00:00:00
acquisition_channel	str	0.0	Paid Social
full_name	str	0.0	Shawn Rose
email	str	0.0	shawn.rose819@proton.me
city	str	0.0	Brownland
state_region	str	0.0	KS
postal_code	str	0.0	75421
segment	str	0.0	Prosumer
birth_year	float64	0.0	1975.0
marketing_consent	bool	0.0	False
has_email	bool	0.0	True
acquired_before_window	bool	0.0	True
acquisition_month	str	0.0	2023-12
age_band	str	0.0	45-54
country	str	0.0	United States
first_order_date	datetime64[ms]	10.6	2024-08-02 00:00:00
last_order_date	datetime64[ms]	10.6	2024-08-02 00:00:00
lifetime_orders	int64	0.0	0
lifetime_net_revenue_usd	float64	0.0	0.0

column	dtype	null %	sample
lifetime_gross_profit_usd	float64	10.7	41.129999999999995
is_repeat_customer	bool	0.0	False
customer_status	str	0.0	Prospect
loyalty_tier	str	0.0	Standard

dim_product — 295 rows

Product dimension: category hierarchy, price tier, lifecycle dates (+ Unknown stub).

column	dtype	null %	sample
product_key	int64	0.0	1
sku_id	str	0.0	SKU-00001
product_name	str	0.0	Rivon Mid-range Smartphones 656
category	str	0.0	Smartphones
subcategory	str	0.0	Mid-range
brand	str	0.0	Rivon
list_price_usd	float64	0.0	399.0
unit_cost_usd	float64	0.3	378.49
launch_date	datetime64[us]	0.3	2025-05-22 00:00:00
discontinue_date	datetime64[us]	83.7	2026-01-29 00:00:00
is_warranty_eligible	bool	0.0	True
weight_kg	float64	0.3	0.519
warranty_months	float64	0.3	12.0
supplier_id	str	0.3	SUP-014
is_active_at_window_end	bool	0.0	True
price_tier	str	0.0	Premium
supplier_key	int64	0.0	14

dim_supplier — 15 rows

Supplier dimension: payment terms band, lead time, reliability band.

column	dtype	null %	sample
supplier_key	int64	0.0	1
supplier_id	str	0.0	SUP-001
supplier_name	str	0.0	Shenzhen Components
country	str	0.0	KR
payment_terms_days	int64	0.0	30
lead_time_days	int64	0.0	21
on_time_rate	float64	0.0	0.905
payment_terms_band	str	0.0	Net 30
reliability_band	category	0.0	Medium

dim_channel — 8 rows

Marketing / acquisition channels grouped Paid / Owned / Earned.

column	dtype	null %	sample
channel_key	int64	0.0	1
channel_name	str	0.0	Paid Search
channel_group	str	0.0	Paid

column	dtype	null %	sample
is_paid	bool	0.0	True

dim_campaign — 584 rows

Distinct marketing campaigns with parsed theme.

column	dtype	null %	sample
campaign_key	int64	0.0	1
campaign	str	0.0	US_PaidSearch_Q32024_Always-On
market_id	str	0.0	US
channel	str	0.0	Paid Search
campaign_theme	str	0.0	Always-On

dim_payment_method — 16 rows

Payment method rate card: MDR, fixed fee, settlement lag, instalment rules.

column	dtype	null %	sample
payment_method_key	int64	0.0	1
method_name	str	0.0	Visa Credit
method_group	str	0.0	Card
card_scheme	str	0.0	Visa
mdr_pct	float64	0.0	0.022
fixed_fee_local_nominal	float64	0.0	0.0
settlement_days	int64	0.0	2
installment_eligible	bool	0.0	True
max_installments	int64	0.0	12
available_markets	str	0.0	US,UK,DE,BR

dim_carrier — 12 rows

Carriers per market with baseline transit days and on-time rate.

column	dtype	null %	sample
carrier_key	int64	0.0	1
carrier_name	str	0.0	UPS
market_id	str	0.0	US
baseline_transit_days	float64	0.0	3.5
baseline_on_time_rate	float64	0.0	0.94

dim_warehouse — 5 rows

Fulfilment centres per market.

column	dtype	null %	sample
warehouse_key	int64	0.0	1
warehouse_id	str	0.0	FC-DAL
warehouse_name	str	0.0	Dallas, TX
market_id	str	0.0	US
region	str	0.0	South

dim_device — 3 rows

Mobile / Desktop / Tablet.

column	dtype	null %	sample
device_key	int64	0.0	1
device	str	0.0	Mobile

dim_return_reason — 7 rows

Return reason codes.

column	dtype	null %	sample
return_reason_key	int64	0.0	1
reason	str	0.0	Defective / not working

dim_ticket_category — 7 rows

Support ticket categories.

column	dtype	null %	sample
ticket_category_key	int64	0.0	1
category	str	0.0	Where is my order

dim_order_status — 4 rows

Order status values.

column	dtype	null %	sample
order_status_key	int64	0.0	1
order_status	str	0.0	delivered

dim_metric — 5 rows

Target metric names.

column	dtype	null %	sample
metric_key	int64	0.0	1
metric	str	0.0	Net Revenue

fact_order_lines — 261,589 rows

One row per order line (product or warranty). Centre of the model.

column	dtype	null %	sample
order_line_id	str	0.0	OL-00153335
order_id	str	0.0	BR-202511-0106878
line_type	str	0.0	product
order_date_key	Int64	0.0	20251107
ship_date_key	Int64	0.0	20251108
delivery_date_key	Int64	0.7	20260307
product_key	int64	0.0	157
customer_key	int64	0.0	95977
market_key	int64	0.0	4
currency_key	int64	0.0	4
channel_key	int64	0.0	2
warehouse_key	int64	0.0	5
payment_method_key	int64	0.0	10
order_status	str	0.0	cancelled
quantity	int64	0.0	1

column	dtype	null %	sample
unit_price_local	float64	0.0	160.99
gross_amount_local	float64	0.0	160.99
discount_amount_local	float64	0.0	0.0
net_amount_local	float64	0.0	160.99
gross_amount_usd	float64	0.0	31.768800660000004
discount_amount_usd	float64	0.0	0.0
net_amount_usd	float64	0.0	31.768800660000004
cogs_usd	float64	0.0	21.7914
gross_profit_usd	float64	0.0	9.977400660000004
returned_qty	int64	0.0	0
is_returned	bool	0.0	False
refund_amount_local	float64	92.2	13.83
refund_amount_usd	float64	0.0	0.0

fact_orders — 159,405 rows

One row per order header with delivery / perfect-order flags.

column	dtype	null %	sample
order_id	str	0.0	US-202407-0000001
order_date_key	Int64	0.0	20240701
ship_date_key	Int64	0.0	20240704
delivery_date_key	Int64	0.7	20240705
customer_key	int64	0.0	34782
market_key	int64	0.0	1
currency_key	int64	0.0	1
channel_key	int64	0.0	2
warehouse_key	int64	0.0	1
carrier_key	int64	0.0	1
payment_method_key	int64	0.0	3
order_status	str	0.0	delivered
device	str	0.0	Mobile
installments	int64	0.0	1
installment_plan	str	0.0	None
units	int64	0.0	2
gross_amount_local	float64	0.0	214.98
discount_amount_local	float64	0.0	16.75
net_amount_local	float64	0.0	198.23
tax_amount_local	float64	0.0	13.88
shipping_fee_local	float64	0.0	0.0
payment_fee_local	float64	0.0	4.67
gross_amount_usd	float64	0.0	214.98
discount_amount_usd	float64	0.0	16.75
net_amount_usd	float64	0.0	198.23
tax_amount_usd	float64	0.0	13.88
shipping_fee_usd	float64	0.0	0.0

column	dtype	null %	sample
payment_fee_usd	float64	0.0	4.67
shipping_cost_usd	float64	0.0	13.36
ship_days	int64	0.0	3
delivery_days	float64	0.7	4.0
is_late	bool	0.0	False
is_on_time	bool	0.0	True
is_delivered	bool	0.0	True
is_cancelled	bool	0.0	False
has_return	bool	0.0	False
is_perfect_order	bool	0.0	True

fact_payment_schedule — 215,541 rows

One row per settlement tranche (order × instalment).

column	dtype	null %	sample
order_id	str	0.0	US-202407-0000001
n_installments	int64	0.0	1
installment_number	int64	0.0	1
installment_plan	str	0.0	None
gross_amount_local	float64	0.0	212.10999999999999
fee_amount_local	float64	0.0	4.67
financing_cost_local	float64	0.0	0.0
net_amount_local	float64	0.0	207.44
market_key	int64	0.0	1
currency_key	int64	0.0	1
order_date_key	Int64	0.0	20240701
due_date_key	Int64	0.0	20240703
settlement_lag_days	int64	0.0	2
gross_amount_usd	float64	0.0	212.10999999999999
fee_amount_usd	float64	0.0	4.67
financing_cost_usd	float64	0.0	0.0
net_amount_usd	float64	0.0	207.44

fact_returns — 20,339 rows

One row per return line.

column	dtype	null %	sample
return_id	str	0.0	RET-0000001
order_id	str	0.0	US-202407-0000020
order_line_id	str	0.0	OL-00000019
return_date_key	Int64	0.0	20240727
product_key	int64	0.0	151
customer_key	int64	0.0	14122
market_key	int64	0.0	1
currency_key	int64	0.0	1
returned_qty	int64	0.0	1

column	dtype	null %	sample
refund_amount_local	float64	0.0	36.99
refund_amount_usd	float64	0.0	36.99
reason	str	0.0	Changed my mind
restocked	bool	0.0	True
condition	str	0.0	sellable

fact_purchase_orders — 13,732 rows

One row per PO line with receipt / due / paid dates.

column	dtype	null %	sample
po_number	str	0.0	PO-000001
supplier_key	int64	0.0	15
product_key	int64	0.0	34
warehouse_key	int64	0.0	1
market_key	int64	0.0	1
order_date_key	Int64	0.0	20240611
expected_receipt_date_key	Int64	0.0	20240627
actual_receipt_date_key	Int64	0.0	20240627
payment_due_date_key	Int64	0.0	20240727
supplier_paid_date_key	Int64	0.0	20240725
qty_ordered	int64	0.0	50
unit_cost_usd	float64	0.0	177.57
po_value_usd	float64	0.0	8878.5
payment_terms_days	int64	0.0	30
late_receipt_days	int64	0.0	0
days_to_pay	int64	0.0	-2

fact_inventory_movement — 268,159 rows

Stock movement ledger with moving-average valuation.

column	dtype	null %	sample
movement_id	str	0.0	MOV-000000001
movement_date_key	Int64	0.0	20250429
product_key	int64	0.0	1
warehouse_key	int64	0.0	3
market_key	int64	0.0	2
movement_type	str	0.0	receipt
quantity	int64	0.0	10
unit_cost_usd	float64	0.0	399.1324
value_usd	float64	0.0	3991.32

fact_inventory_snapshot — 26,532 rows

Month-end stock per SKU×warehouse, derived from the ledger.

column	dtype	null %	sample
snapshot_date_key	int64	0.0	20240731
month_year	str	0.0	2024-07
sku_id	str	0.0	SKU-00002

column	dtype	null %	sample
product_key	int64	0.0	2
warehouse_key	int64	0.0	1
market_key	int64	0.0	1
units_on_hand	int64	0.0	105
moving_avg_cost_usd	float64	0.0	162.7105
inventory_value_usd	float64	0.0	17084.6
units_in_transit	int64	0.0	0
avg_weekly_units_sold	float64	0.0	2.62
weeks_of_cover	float64	9.2	40.0
raw_snapshot_units	int64	0.0	105
snapshot_variance_units	int64	0.0	0

fact_marketing_spend — 10,935 rows

Daily spend / impressions / clicks by market-channel-campaign.

column	dtype	null %	sample
date_key	Int64	0.0	20240701
market_key	int64	0.0	1
channel_key	int64	0.0	1
campaign_key	int64	0.0	1
impressions	int64	0.0	18632
clicks	int64	0.0	993
cost_local	float64	0.0	821.33
cost_usd	float64	0.0	821.33

fact_web_traffic_daily — 34,950 rows

Daily sessions & funnel by market-channel-device.

column	dtype	null %	sample
date_key	Int64	0.0	20240701
market_key	int64	0.0	1
channel_key	int64	0.0	6
device	str	0.0	Desktop
sessions	int64	0.0	71
users	int64	0.0	61
pageviews	int64	0.0	344
bounces	int64	0.0	33
add_to_carts	int64	0.0	6
carts_created	int64	0.0	3
transactions	int64	0.0	2
had_data_anomaly	bool	0.0	False

fact_support_tickets — 22,257 rows

One row per support ticket.

column	dtype	null %	sample
ticket_id	str	0.0	TCK-0000001
order_id	str	0.0	US-202407-0000003

column	dtype	null %	sample
created_date_key	Int64	0.0	20240708
resolved_date_key	Int64	0.0	20240708
market_key	int64	0.0	1
customer_key	int64	0.0	40095
category	str	0.0	Return / refund
priority	str	0.0	Medium
resolution_hours	float64	0.0	6.846666666666667
first_contact_resolution	bool	0.0	False
csat	int64	0.0	5

fact_exchange_rate — 4,384 rows

Daily rate_to_usd / rate_from_usd per currency.

column	dtype	null %	sample
date_key	Int64	0.0	20240101
currency_key	int64	0.0	1
currency_code	str	0.0	USD
rate_to_usd	float64	0.0	1.0
rate_from_usd	float64	0.0	1.0

fact_target — 360 rows

Monthly plan value per market and metric.

column	dtype	null %	sample
month_date_key	Int64	0.0	20250701
market_key	int64	0.0	4
metric	str	0.0	Net Revenue
target_value	float64	0.0	140772.51

04 — ETL Pipeline

The pipeline turns one seed into a full messy-source dataset and a clean star schema. Four stages, each a standalone script, chained by `run_pipeline.py`.

```
config.py → 01_generate_raw.py → 02_clean_stage.py → 03_build_curated.py → dq_checks.py
           data/raw/           data/staging/       data/curated/       data/quality/dq_report.md
```

Run it all:

```
cd etl
python -m pip install -r requirements.txt
python run_pipeline.py           # 01 → 02 → 03 → 04, stops on first failure
python run_pipeline.py --from 3  # re-run from a later stage
```

Deterministic: one `SEED` in `config.py` drives every random draw, so a re-run reproduces byte-identical output.

`config.py` — the single control panel

Everything tunable lives here: the 24-month window and YoY split, the four markets and their go-live dates, the product taxonomy with per-category price ranges / margins / return rates, order-volume ramps, seasonality and promo calendar, customer and returning-buyer behaviour, channel economics, the full payment-method rate card, supplier terms, logistics SLAs, and the catalogue of data-quality noise to inject (`DQ`).

Stage 01 — `01_generate_raw.py` (bronze)

Generates 12 "source system" extracts into `data/raw/`. Build order and the dependencies between them:

#	Output	Notes
1	<code>fx_rates.csv</code>	Daily USD-per-unit for EUR/GBP/BRL as a mean-reverting random walk around anchors. Written <code>d/m/Y</code> , USD row omitted.
2	<code>suppliers.csv</code>	15 suppliers: country, <code>payment_terms_days</code> (Net 30/45/60), <code>lead_time_days</code> , <code>on_time_rate</code> .
3	<code>erp_products.xlsx</code>	300 SKUs across 3 sheets (<code>products</code> , <code>cost_history</code> , <code>price_history</code>). Prices skewed to the low end of each band; unit cost = price × (1 – margin) then a mean-reverting monthly walk . Injected mess: category spelled 3 ways on ~30% of rows, lowercased brands, "\$ 1,234.56" strings, <code>m/d/Y</code> dates, and ~6 SKUs omitted from the master (they still sell → late-arriving dimension).
4	Orders	The core generator. Month-by-month per market: derive order count from the ramp × seasonality × promo/Black-Friday day weights; split into new vs returning customers (returning sampled with a recency×frequency weight); build each basket (popularity-weighted SKU pick ~ 1/price, correlated accessory add-on, warranty attach on eligible items); attach channel, device, warehouse, carrier, ship/promise/deliver dates, payment method and instalment plan. Emits <code>oms_orders.csv</code> , <code>oms_order_lines.csv</code> and seeds every downstream table. Injected mess: 5 date formats incl. Excel serials, money strings, ~0.5% duplicate lines, ~0.3% blank customer id, ~0.2% negative quantities.
5	CRM	<code>crm_customers.csv</code> (utf-8) + <code>crm_customers_br.csv</code> (Latin-1 , to be detected). ~2% null e-mail, ~1.5% duplicate rows, mixed casing, trailing spaces.
6	<code>psp_transactions.csv</code>	One auth per order + ~1% unmatched junk + ~0.4% of orders missing. Fee column is deliberately mislabeled <code>mdr_pct</code> while holding a currency amount.
7	<code>procurement_purchase_orders.csv</code>	Reorder loop per SKU×market with real movement: when projected on-hand drops below <code>TARGET_WEEKS_OF_COVER</code> , raise a PO (rounded to a case pack) dated back by the supplier lead time; late receipts for unreliable suppliers; <code>payment_due</code> = <code>receipt</code> + <code>terms</code> ; <code>supplier_paid</code> = <code>due</code> ± few days.
8	WMS	<code>wms_inventory_movements.csv</code> — a ledger of <code>adjustment</code> (opening), <code>receipt</code> (from POs) and <code>sale</code> (from order lines). <code>wms_inventory_snapshots.csv</code> — month-end balances taken from that ledger, then ~3% of rows perturbed so the snapshot and the ledger disagree.
9	<code>returns.csv</code>	~1 line per return with reason, restock flag, <code>d/m/Y</code> dates.
10	<code>web_analytics_daily.csv</code>	Sessions reverse-engineered from orders and a channel×device conversion rate; funnel columns; ~1.5% of dates dropped; a few negative-session rows.
11	Marketing	<code>google_ads_report.csv</code> (Search + Display), <code>meta_ads_export.csv</code> (Social, different column names & date format), <code>other_marketing_spend.csv</code> (affiliate commission + email tooling, monthly). Budget sized to hit a target MER, then split by channel.
12	<code>support_tickets.csv</code>	~14% of orders raise a ticket: category, priority, created/resolved timestamps, FCR flag, CSAT.

#	Output	Notes
—	<code>carrier_tracking.json</code>	Nested <code>{shipments:[{events:[...]}]}</code> for a ~9% sample; status vocabulary inconsistent on purpose (<code>delivered</code> / <code>DELIVERED</code> / <code>Delivery complete</code>).
—	<code>finance_targets.csv</code>	Monthly plan per market for 5 metrics, set so actuals land ~90–110% of plan.

Stage 02 — `02_clean_stage.py` (silver)

Reads `data/raw/*`, writes typed Parquet to `data/staging/` (`stg_*`), and records every fix as a Markdown fragment in `data/quality/_fragments/`.

Key routines:

- `parse_money` — locale-tolerant: `$ 1,299.00`, `1.299,00`, `R$ 1.299,00`, blanks, floats.
- `parse_dt` — tries 8 explicit formats then Excel serial numbers then a loose fallback.
- **Products** — map the 27 observed category spellings to 7 canonical values via a lookup built from `config.CATEGORY_TYPOS`; title-case brands; parse price/dates; drop duplicate SKUs.
- **Customers** — probe the regional file's encoding (`utf-8` ? else `latin-1`), concat, drop exact-duplicate rows, then de-dup on `customer_id`; normalise strings; flag missing e-mail.
- **Orders / order lines** — parse, type, drop duplicate lines, move non-positive-qty lines to `stg_order_line_rejects`, flag orphan orders/lines and SKUs missing from the master.
- **PSP** — rename the mislabelled column to `reported_fee_local`; reconcile `order_ref` against orders (matched / unmatched both reported).
- **Inventory** — reconcile the raw month-end snapshot against a running balance rebuilt from the movement ledger; report the variance.
- **Marketing** — union the three sources into one long `stg_marketing_spend` (`date · market · channel · campaign · impressions · clicks · cost_local · currency · source_system`); the monthly `other` rows are spread evenly across their days.
- **Carrier JSON** — flatten to `stg_carrier_events` (one row per event) and `stg_shipments` (one row per order with created/in-transit/out-for-delivery/delivered timestamps); collapse the status vocabulary.
- **FX** — parse, add `USD = 1.0` for every calendar day, forward/back-fill to the full `dim_date` span.

Stage 03 — `03_build_curated.py` (gold)

Builds the Kimball star schema in `data/curated/` (Parquet **and** CSV per table).

Dimensions

`dim_date` (fiscal year starts 1 July; promo & Black-Friday flags; `yoy_period = CY/PY`), `dim_market` (`live_date`, `is_comparable_base`), `dim_currency`, `dim_customer` (two-pass: attributes first, then lifetime orders / revenue / status / loyalty tier from the facts), `dim_product` (price tier by in-category quantile, + an Unknown stub with `product_key = -1`), `dim_supplier`, `dim_channel`, `dim_campaign`, `dim_payment_method`, `dim_carrier`, `dim_warehouse`, and small dims for device / return reason / ticket category / order status / target metric.

Techniques worth calling out

- **Currency conversion** — `FX` helper holds a `{(date, ccy): rate}` map; every `*_local` money column gets a materialised `*_usd` sibling at the transaction-date rate.
- **Moving-average COGS** — `build_cost_curves` walks each SKU's receipts (opening adjustment valued at first known cost, then every PO receipt at its PO unit cost) and keeps a running weighted-average. `cost_asof(sku, date)` is a binary search on that step function. Sale lines take `qty × cost_asof`; warranty lines take a cost ratio.
- **fact_payment_schedule** — each non-cancelled order is exploded into one row per settlement tranche: à-vista = a single row at `order_date + settlement_days`; instalments = *N* monthly rows. Fees are pro-rated; interest-free (merchant-funded) plans also accrue a monthly financing cost on the declining balance.
- **fact_inventory_snapshot** — **derived from the movement ledger**, not copied from the raw snapshot: month-end running balance per SKU×warehouse, valued at the moving-average cost, plus units in transit (open POs) and weeks-of-cover from trailing 8-week sales. The raw snapshot value and its variance are carried alongside for the DQ story.
- **Comparable base** — `dim_market.is_comparable_base` flags markets live for the whole CY+PY span (US). The report also recomputes this dynamically in DAX.

Facts

`fact_order_lines` (line grain — the centre of the model), `fact_orders` (header grain, with delivery / perfect-order flags), `fact_payment_schedule`, `fact_returns`, `fact_purchase_orders`, `fact_inventory_movement`, `fact_inventory_snapshot`, `fact_marketing_spend`, `fact_web_traffic_daily`, `fact_support_tickets`, `fact_exchange_rate`, `fact_target`.

Grain, columns and relationships are catalogued in [05 data model.md](#) and [03 data dictionary.md](#).

Stage 04 — `dq_checks.py` (gate)

Structural and business assertions over the curated layer, split **HARD** (fails the build) vs **SOFT** (reported):

- no null surrogate keys in the fact tables;
- referential integrity — every fact FK resolves to a dimension key (or the `-1` stub);
- every fact date is present in `dim_date` ;
- accounting identities — `net = gross - discount` on every line; order lines roll up to the order header; payment schedule sums back to the order total;
- `comparable base = US only` ; FX `USD = 1.0` ; no non-positive rates;
- sanity bands — order count, AOV, blended gross margin, share of below-cost lines.

Output: `data/quality/dq_report.md` (this gate's table + the stage-02 cleaning fragments). Non-zero exit on any HARD failure, which also halts `run_pipeline.py` .

05 — Data Model (Curated Star Schema)

The curated layer is a Kimball-style star: one row per business event in the fact tables, conformed dimensions with integer surrogate keys, a real date dimension, and every monetary value available in both local currency and USD.

Power BI connects to `data/curated/*.parquet` through a single parameter (`pDataFolder`).

1. Bus matrix — which dimension touches which fact

Dimension ▾ / Fact ▾	date	market	customer	product	supplier	channel	campaign	payment_method	carrier	warehouse	currency
fact_order_lines	• (order/ship/deliver)	•	•	•		•		•		•	•
fact_orders	• (order/ship/deliver)	•	•			•		•	•	•	•
fact_payment_schedule	• (order/due)	•									•
fact_returns	• (return)	•	•	•							•
fact_purchase_orders	• (order/receipt/due/paid)	•		•	•					•	
fact_inventory_movement	• (movement)	•		•						•	
fact_inventory_snapshot	• (month-end)	•		•						•	
fact_marketing_spend	•	•				•	•				
fact_web_traffic_daily	•	•				•					
fact_support_tickets	• (created/resolved)	•	•								
fact_exchange_rate	•										•
fact_target	• (month)	•									

All date roles join to the **single** `dim_date` (Power BI: one active relationship on `order_date_key` ; `ship_date_key` , `delivery_date_key` , etc. are inactive relationships activated with `USERELATIONSHIP` inside specific measures).

2. Fact tables — grain & measures

fact_order_lines (centre of the model)

Grain: one row per order line — an order × product, or a `warranty` add-on line. **Keys:** `order_date_key` , `ship_date_key` , `delivery_date_key` , `product_key` , `customer_key` , `market_key` , `currency_key` , `channel_key` , `warehouse_key` , `payment_method_key` ; degenerate `order_id` , `order_line_id` , `line_type` , `order_status` . **Measures:** `quantity` , `unit_price_local` , `gross_amount_(local|usd)` , `discount_amount_(local|usd)` , `net_amount_(local|usd)` , `cogs_usd` (moving-average), `gross_profit_usd` , `returned_qty` , `is_returned` , `refund_amount_(local|usd)` .

fact_orders

Grain: one row per order header. **Keys:** `order/ship/delivery date` , `customer_key` , `market_key` , `currency_key` , `channel_key` , `warehouse_key` , `carrier_key` , `payment_method_key` ; degenerate `order_id` , `order_status` , `device` , `installments` , `installment_plan` . **Measures:** `units` , `gross/discount/net/tax/shipping_fee/payment_fee` in `_local` and `_usd` , `shipping_cost_usd` , `ship_days` , `delivery_days` ; flags `is_late` , `is_on_time` , `is_delivered` , `is_cancelled` , `has_return` , `is_perfect_order` .

fact_payment_schedule

Grain: one row per settlement tranche (`order_id` × `installment_number`). **Keys:** `order_date_key` , `due_date_key` , `market_key` , `currency_key` . **Measures:** `n_installments` , `installment_plan` , `settlement_lag_days` , `gross/fee/financing_cost/net amount` in `_local` and `_usd` . Cash **received** in a period = SUM(`net_amount_usd`) filtered by `due_date_key` ; revenue **booked** = `fact_order_lines` filtered by `order_date_key` . The gap is the DSO story.

fact_returns

Grain: one return line. `return_date_key` , `product_key` , `customer_key` , `market_key` , `currency_key` ; `returned_qty` , `refund_amount_(local|usd)` , `reason` , `restocked` , `condition` .

fact_purchase_orders

Grain: one PO line (`po_number` × `SKU`). `order_date_key` , `expected/actual_receipt_date_key` , `payment_due_date_key` , `supplier_paid_date_key` , `supplier_key` , `product_key` , `warehouse_key` , `market_key` . **Measures:** `qty_ordered` , `unit_cost_usd` , `po_value_usd` , `payment_terms_days` ,

`late_receipt_days`, `days_to_pay`. Accounts payable open at a date = POs with `actual_receipt ≤ date < supplier_paid`.

fact_inventory_movement

Grain: one stock movement. `movement_date_key`, `product_key`, `warehouse_key`, `market_key`; `movement_type` (`adjustment` / `receipt` / `sale`), signed `quantity`, `unit_cost_usd` (moving-average at the date), `value_usd`.

fact_inventory_snapshot (derived from the ledger)

Grain: month-end × SKU × warehouse. `snapshot_date_key`, `month_year`, `product_key`, `warehouse_key`, `market_key`. **Measures:** `units_on_hand`, `moving_avg_cost_usd`, `inventory_value_usd`, `units_in_transit`, `avg_weekly_units_sold`, `weeks_of_cover`, plus `raw_snapshot_units` and `snapshot_variance_units` for the DQ narrative.

fact_marketing_spend

Grain: date × market × channel × campaign. `impressions`, `clicks`, `cost_local`, `cost_usd`.

fact_web_traffic_daily

Grain: date × market × channel × device. `sessions`, `users`, `pageviews`, `bounces`, `add_to_carts`, `carts_created`, `transactions`, `had_data_anomaly`.

fact_support_tickets

Grain: one ticket. `created_date_key`, `resolved_date_key`, `market_key`, `customer_key`, `category`, `priority`, `resolution_hours`, `first_contact_resolution`, `csat`.

fact_exchange_rate

Grain: date × currency. `rate_to_usd`, `rate_from_usd`. Used by currency-aware measures and the reporting-currency selector.

fact_target

Grain: month × market × metric. `target_value`. `metric` ∈ {Net Revenue, Orders, New Customers, Blended CAC, Gross Margin %}.

3. Dimensions — notable attributes

Dimension	Key	Attributes used in the report
dim_date	<code>date_key</code> (yyyymmdd)	<code>date</code> , <code>month_year</code> , <code>quarter_name</code> , <code>year</code> , <code>day_name</code> , <code>is_weekend</code> , <code>fiscal_year</code> / <code>fiscal_quarter</code> / <code>fiscal_period</code> (FY starts 1 Jul), <code>is_promo_period</code> , <code>promo_name</code> , <code>is_black_friday_week</code> , <code>in_extract_window</code> , <code>yoy_period</code> (CY/PY)
dim_market	<code>market_key</code>	<code>market_id</code> , <code>market_name</code> , <code>region</code> , <code>currency_code</code> , <code>live_date</code> , <code>is_comparable_base</code>
dim_currency	<code>currency_key</code>	<code>currency_code</code> , <code>currency_name</code> , <code>currency_symbol</code> , <code>is_reporting_currency</code>
dim_customer	<code>customer_key</code> (-1 = Unknown)	<code>market_id</code> , <code>acquisition_channel</code> , <code>acquisition_month</code> , <code>acquired_before_window</code> , <code>segment</code> , <code>age_band</code> , <code>loyalty_tier</code> , <code>marketing_consent</code> , <code>has_email</code> , <code>country</code> , <code>state_region</code> , <code>city</code> , <code>first_order_date</code> , <code>last_order_date</code> , <code>lifetime_orders</code> , <code>lifetime_net_revenue_usd</code> , <code>is_repeat_customer</code> , <code>customer_status</code> (Prospect/Active/Lapsed/Churned)
dim_product	<code>product_key</code> (-1 = Unknown/late-arriving)	<code>sku_id</code> , <code>product_name</code> , <code>category</code> , <code>subcategory</code> , <code>brand</code> , <code>list_price_usd</code> , <code>price_tier</code> , <code>launch_date</code> , <code>discontinue_date</code> , <code>is_active_at_window_end</code> , <code>is_warranty_eligible</code> , <code>warranty_months</code> , <code>weight_kg</code> , <code>supplier_key</code>
dim_supplier	<code>supplier_key</code>	<code>supplier_name</code> , <code>country</code> , <code>payment_terms_days</code> , <code>payment_terms_band</code> (Net 30/45/60), <code>lead_time_days</code> , <code>on_time_rate</code> , <code>reliability_band</code>
dim_channel	<code>channel_key</code>	<code>channel_name</code> , <code>channel_group</code> (Paid/Owned/Earned), <code>is_paid</code>
dim_campaign	<code>campaign_key</code>	<code>campaign_name</code> , <code>market_id</code> , <code>channel</code> , <code>campaign_theme</code>
dim_payment_method	<code>payment_method_key</code> (-1)	<code>method_name</code> , <code>method_group</code> , <code>card_scheme</code> , <code>mdr_pct</code> , <code>fixed_fee_local_nominal</code> , <code>settlement_days</code> , <code>installment_eligible</code> , <code>max_installments</code> , <code>available_markets</code>
dim_carrier	<code>carrier_key</code>	<code>carrier_name</code> , <code>market_id</code> , <code>baseline_transit_days</code> , <code>baseline_on_time_rate</code>
dim_warehouse	<code>warehouse_key</code>	<code>warehouse_id</code> , <code>warehouse_name</code> , <code>market_id</code> , <code>region</code>
small dims		<code>dim_device</code> , <code>dim_return_reason</code> , <code>dim_ticket_category</code> , <code>dim_order_status</code> , <code>dim_metric</code>

4. Cross-cutting model decisions

Multi-currency. Transactions are captured in local currency (`*_local`) with a `currency_key`, and every one carries a materialised `*_usd` at the order-date rate. `fact_exchange_rate` additionally lets DAX convert to any reporting currency on the fly; a disconnected `Reporting Currency` slicer drives

that. Default view is USD.

Total vs Comparable Base. `dim_market.is_comparable_base` (US only) gives the static like-for-like cut. A dynamic DAX version recomputes the eligible market set from the visual's date range, so the concept survives any filter. $\text{Expansion Contribution} = \text{Total} - \text{Comparable Base}$.

Moving-average COGS. `cogs_usd` on each sale line reflects the weighted-average purchase cost of that SKU at sale time (rebuilt from PO receipts), so margin moves with real sourcing cost, not a static list cost.

Inventory snapshot is derived, not trusted. `fact_inventory_snapshot` is rebuilt from the movement ledger; the raw WMS snapshot is kept only as `raw_snapshot_units` / `snapshot_variance_units` to quantify system drift.

Working-capital timing. Three clocks: goods received → paid supplier (`fact_purchase_orders`), stock held (`fact_inventory_snapshot`), order booked → cash settled (`fact_payment_schedule`). Together they give DIO, DPO, DSO and the cash conversion cycle, and a single company cash-flow timeline.

Unknown members. `product_key = -1` and `customer_key = -1` stubs absorb late-arriving / missing natural keys so fact rows are never dropped and totals always tie.

5. Power BI relationships (all single-direction, many-to-one from fact to dim)

<code>dim_date[date_key]</code>	<code>1 →* fact_order_lines[order_date_key]</code>	(active)
<code>dim_date[date_key]</code>	<code>1 →* fact_order_lines[ship_date_key]</code>	(inactive)
<code>dim_date[date_key]</code>	<code>1 →* fact_order_lines[delivery_date_key]</code>	(inactive)
<code>dim_market[market_key]</code>	<code>1 →* fact_order_lines[market_key]</code>	
<code>dim_customer[customer_key]</code>	<code>1 →* fact_order_lines[customer_key]</code>	
<code>dim_product[product_key]</code>	<code>1 →* fact_order_lines[product_key]</code>	
<code>dim_channel[channel_key]</code>	<code>1 →* fact_order_lines[channel_key]</code>	
<code>dim_currency[currency_key]</code>	<code>1 →* fact_order_lines[currency_key]</code>	
<code>dim_payment_method[...key]</code>	<code>1 →* fact_orders[payment_method_key]</code>	
<code>dim_carrier[carrier_key]</code>	<code>1 →* fact_orders[carrier_key]</code>	
<code>dim_warehouse[warehouse_key]</code>	<code>1 →* fact_orders[warehouse_key]</code>	<code>fact_purchase_orders</code> , <code>fact_inventory_*</code>
<code>dim_supplier[supplier_key]</code>	<code>1 →* fact_purchase_orders[supplier_key]</code>	
<code>dim_campaign[campaign_key]</code>	<code>1 →* fact_marketing_spend[campaign_key]</code>	
<code>dim_date[date_key]</code>	<code>1 →* every other fact's primary date key</code>	

`fact_orders` and `fact_order_lines` are **not** related to each other — they are two grains of the same process and are sliced by shared dimensions, never joined.

06 — DAX Measure Library

Generated by `etl/gen_dax_doc.py` from `etl/pbi_measures.py`, which is also the source `etl/gen_semantic_model.py` writes into the `_Measures` table of the TMDL model.

167 measures in **14 display folders**. All monetary measures are USD (the reporting currency); `*_local` columns remain on detail tables. Sales measures exclude cancelled orders. Time intelligence runs on `dim_date[date]` (contiguous daily, auto date/time disabled).

Conventions

Suffix / prefix	Meaning
(USD)	Amount in US dollars (reporting currency)
(Rpt)	Amount converted to the currency chosen in the <i>Reporting currency</i> slicer
PY	Same measure, same-period-last-year
YoY / YoY %	Current – PY, and that over PY
(Comparable Base)	Restricted to markets live for the whole CY+PY span (US) — the like-for-like cut
(Expansion)	Total – Comparable Base

Key definitions

- Net Revenue** = Gross Revenue – Discounts – Returns (`Gross Sales` – `Returns`).
- Gross Profit** = Net Revenue – COGS (moving-average cost). Returns reduce revenue by the refund; the COGS of restocked goods is not added back, so margin is a conservative estimate.
- Contribution Margin** = Gross Profit – Marketing Spend – Shipping Cost – Total Payment Cost.
- Cash Conversion Cycle** = DIO + DSO – DPO, from `fact_inventory_snapshot`, `fact_payment_schedule` and `fact_purchase_orders` respectively.

00 Selectors

Exec Insight

```
Exec Insight =
VAR _nr    = [Net Revenue (USD)]
VAR _yoy   = [Net Revenue YoY %]
VAR _yoyC  = [Net Revenue YoY % (Comparable Base)]
VAR _gm    = [Gross Margin %]
VAR _cm    = [Contribution Margin %]
VAR _cmT   = [Contribution Margin % (Trailing 12M)]
VAR _cac   = [Blended CAC (USD)]
VAR _rep   = [Repeat Purchase Rate %]
VAR _ot    = [On-Time Delivery %]
VAR _tgt   = [Net Revenue vs Target %]
VAR _dir   = IF ( _cmT >= -0.01, "at break-even", IF ( _cmT >= -0.05, "closing on break-even", "still loss-making" ) )
RETURN
    "Net revenue " & FORMAT ( _nr / 1000000, "$#,0.0" ) & "M"
    & IF ( ISBLANK ( _yoy ), "", ", " & FORMAT ( _yoy, "+0.0%;-0.0%" ) & " YoY ( " & FORMAT ( _yoyC, "+0.0%;-0.0%" ) & " like-for-li"
    & IF ( ISBLANK ( _tgt ), "", ", " & FORMAT ( _tgt, "+0.0%;-0.0%" ) & " vs plan" ) & ". "
    & "Gross margin " & FORMAT ( _gm, "0.0%" ) & "; contribution margin " & FORMAT ( _cm, "0.0%" )
    & " ( " & FORMAT ( _cmT, "0.0%" ) & " trailing-12m, " & _dir & "). "
    & "Blended CAC " & FORMAT ( _cac, "$#,0" ) & ", repeat rate " & FORMAT ( _rep, "0.0%" )
    & ", on-time delivery " & FORMAT ( _ot, "0.0%" ) & ". "
```

Selected Reporting Currency

```
Selected Reporting Currency =
SELECTEDVALUE('Reporting Currency'[Currency], "USD")
```

FX Rate to Reporting Currency · **format** `#,0.0000`

```

FX Rate to Reporting Currency =
VAR _ccy = [Selected Reporting Currency]
RETURN
IF (
    _ccy = "USD",
    1,
    CALCULATE (
        AVERAGE ( fact_exchange_rate[rate_from_usd] ),
        fact_exchange_rate[currency_code] = _ccy,
        REMOVEFILTERS ( dim_currency )
    )
)

```

01 Sales

Gross Revenue (USD) · format "\$"#,0

```

Gross Revenue (USD) =
CALCULATE ( SUM ( fact_order_lines[gross_amount_usd] ), KEEPFILTERS ( fact_order_lines[order_status] <> "cancelled" ) )

```

Discounts (USD) · format "\$"#,0

```

Discounts (USD) =
CALCULATE ( SUM ( fact_order_lines[discount_amount_usd] ), KEEPFILTERS ( fact_order_lines[order_status] <> "cancelled" ) )

```

Returns (USD) · format "\$"#,0

```

Returns (USD) =
CALCULATE ( SUM ( fact_order_lines[refund_amount_usd] ), KEEPFILTERS ( fact_order_lines[order_status] <> "cancelled" ) )

```

Gross Sales (USD) · format "\$"#,0

```

Gross Sales (USD) =
CALCULATE ( SUM ( fact_order_lines[net_amount_usd] ), KEEPFILTERS ( fact_order_lines[order_status] <> "cancelled" ) )

```

Net Revenue (USD) · format "\$"#,0

```

Net Revenue (USD) =
[Gross Sales (USD)] - [Returns (USD)]

```

COGS (USD) · format "\$"#,0

```

COGS (USD) =
CALCULATE ( SUM ( fact_order_lines[cogs_usd] ), KEEPFILTERS ( fact_order_lines[order_status] <> "cancelled" ) )

```

Returns COGS Recovered (USD) · format "\$"#,0

```

Returns COGS Recovered (USD) =
SUM ( fact_returns[cogs_recovered_usd] )

```

Gross Profit (USD) · format "\$"#,0

```

Gross Profit (USD) =
[Net Revenue (USD)] - [COGS (USD)] + [Returns COGS Recovered (USD)]

```

Gross Margin % · format 0.0%

```

Gross Margin % =
DIVIDE ( [Gross Profit (USD)], [Net Revenue (USD)] )

```

Orders · format #,0

```
Orders =  
CALCULATE ( DISTINCTCOUNT ( fact_orders[order_id] ), KEEPFILTERS ( fact_orders[is_cancelled] = FALSE ) )
```

Units Sold · **format** #,0

```
Units Sold =  
CALCULATE ( SUM ( fact_order_lines[quantity] ), KEEPFILTERS ( fact_order_lines[order_status] <> "cancelled" ), KEEPFILTERS ( fact_o
```

Average Order Value · **format** "\$"#,0.00

```
Average Order Value =  
DIVIDE ( [Net Revenue (USD)], [Orders] )
```

Units per Order · **format** #,0.0

```
Units per Order =  
DIVIDE ( [Units Sold], [Orders] )
```

Discount Rate % · **format** 0.0%

```
Discount Rate % =  
DIVIDE ( [Discounts (USD)], [Gross Revenue (USD)] )
```

Warranty Revenue (USD) · **format** "\$"#,0

```
Warranty Revenue (USD) =  
CALCULATE ( [Gross Sales (USD)], KEEPFILTERS ( fact_order_lines[line_type] = "warranty" ) )
```

Product Revenue (USD) · **format** "\$"#,0

```
Product Revenue (USD) =  
CALCULATE ( [Net Revenue (USD)], KEEPFILTERS ( fact_order_lines[line_type] = "product" ) )
```

Warranty Attach Rate % · **format** 0.0%

```
Warranty Attach Rate % =  
VAR _warr = CALCULATE ( DISTINCTCOUNT ( fact_order_lines[order_id] ), fact_order_lines[line_type] = "warranty" )  
RETURN DIVIDE ( _warr, [Orders] )
```

Contribution Margin (USD) · **format** "\$"#,0

```
Contribution Margin (USD) =  
[Gross Profit (USD)] - [Marketing Spend (USD)] - [Shipping Cost (USD)] - [Total Payment Cost (USD)]
```

Contribution Margin % · **format** 0.0%

```
Contribution Margin % =  
DIVIDE ( [Contribution Margin (USD)], [Net Revenue (USD)] )
```

CM Bridge Value (USD) · **format** "\$"#,0

```
CM Bridge Value (USD) =  
SWITCH (   
    SELECTEDVALUE ( 'PL Line'[Line] ),  
    "Gross Profit", [Gross Profit (USD)],  
    "Marketing", -[Marketing Spend (USD)],  
    "Shipping", -[Shipping Cost (USD)],  
    "Payment Fees", -[Payment Fees (USD)],  
    "Financing", -[Financing Cost Interest-Free (USD)],
```

```
BLANK ()
)
```

Contribution Margin % (Trailing 12M) · format 0.0%

```
Contribution Margin % (Trailing 12M) =
VAR _p = DATESINPERIOD ( dim_date[date], MAX ( dim_date[date] ), -12, MONTH )
RETURN DIVIDE (
    CALCULATE ( [Contribution Margin (USD)], _p ),
    CALCULATE ( [Net Revenue (USD)], _p )
)
```

Rev Bridge Value (USD) · format "\$"#,0

```
Rev Bridge Value (USD) =
SWITCH (
    SELECTEDVALUE ( 'Revenue Bridge'[Line] ),
    "Plan", CALCULATE ( [Net Revenue Target (USD)], REMOVEFILTERS ( dim_market ) ),
    "United States", CALCULATE ( [Net Revenue vs Target (USD)], REMOVEFILTERS ( dim_market ), dim_market[market_id] = "US" ),
    "United Kingdom", CALCULATE ( [Net Revenue vs Target (USD)], REMOVEFILTERS ( dim_market ), dim_market[market_id] = "UK" ),
    "Germany", CALCULATE ( [Net Revenue vs Target (USD)], REMOVEFILTERS ( dim_market ), dim_market[market_id] = "DE" ),
    "Brazil", CALCULATE ( [Net Revenue vs Target (USD)], REMOVEFILTERS ( dim_market ), dim_market[market_id] = "BR" ),
    BLANK ()
)
```

02 Time Intelligence

Net Revenue PY (USD) · format "\$"#,0

```
Net Revenue PY (USD) =
CALCULATE ( [Net Revenue (USD)], SAMEPERIODLASTYEAR ( dim_date[date] ) )
```

Net Revenue YoY (USD) · format "\$"#,0

```
Net Revenue YoY (USD) =
[Net Revenue (USD)] - [Net Revenue PY (USD)]
```

Net Revenue YoY % · format 0.0%

```
Net Revenue YoY % =
DIVIDE ( [Net Revenue YoY (USD)], [Net Revenue PY (USD)] )
```

Orders PY · format #,0

```
Orders PY =
CALCULATE ( [Orders], SAMEPERIODLASTYEAR ( dim_date[date] ) )
```

Orders YoY % · format 0.0%

```
Orders YoY % =
DIVIDE ( [Orders] - [Orders PY], [Orders PY] )
```

AOV PY · format "\$"#,0.00

```
AOV PY =
CALCULATE ( [Average Order Value], SAMEPERIODLASTYEAR ( dim_date[date] ) )
```

AOV YoY % · format 0.0%

```
AOV YoY % =
DIVIDE ( [Average Order Value] - [AOV PY], [AOV PY] )
```

Gross Profit PY (USD) · format "\$"#,0

```
Gross Profit PY (USD) =  
CALCULATE ( [Gross Profit (USD)], SAMEPERIODLASTYEAR ( dim_date[date] ) )
```

Gross Margin % PY · format 0.0%

```
Gross Margin % PY =  
CALCULATE ( [Gross Margin %], SAMEPERIODLASTYEAR ( dim_date[date] ) )
```

Gross Margin % Delta (pp) · format "#,0.0

```
Gross Margin % Delta (pp) =  
( [Gross Margin %] - [Gross Margin % PY] ) * 100
```

Net Revenue YTD (USD) · format "\$"#,0

```
Net Revenue YTD (USD) =  
CALCULATE ( [Net Revenue (USD)], DATESYTD ( dim_date[date], "06-30" ) )
```

Net Revenue MAT (USD) · format "\$"#,0

```
Net Revenue MAT (USD) =  
CALCULATE ( [Net Revenue (USD)], DATESINPERIOD ( dim_date[date], MAX ( dim_date[date] ), -12, MONTH ) )
```

Net Revenue Prev Month (USD) · format "\$"#,0

```
Net Revenue Prev Month (USD) =  
CALCULATE ( [Net Revenue (USD)], DATEADD ( dim_date[date], -1, MONTH ) )
```

Net Revenue MoM % · format 0.0%

```
Net Revenue MoM % =  
DIVIDE ( [Net Revenue (USD)] - [Net Revenue Prev Month (USD)], [Net Revenue Prev Month (USD)] )
```

Net Revenue 3M Avg (USD) · format "\$"#,0

```
Net Revenue 3M Avg (USD) =  
AVERAGEX ( DATESINPERIOD ( dim_date[date], MAX ( dim_date[date] ), -3, MONTH ), [Net Revenue (USD)] )
```

03 Comparable Base (like-for-like)

Net Revenue (Comparable Base) (USD) · format "\$"#,0

```
Net Revenue (Comparable Base) (USD) =  
CALCULATE ( [Net Revenue (USD)], KEEPFILTERS ( dim_market[is_comparable_base] = TRUE ) )
```

Net Revenue (Expansion) (USD) · format "\$"#,0

```
Net Revenue (Expansion) (USD) =  
[Net Revenue (USD)] - [Net Revenue (Comparable Base) (USD)]
```

Net Revenue (Comparable Base) PY (USD) · format "\$"#,0

```
Net Revenue (Comparable Base) PY (USD) =  
CALCULATE ( [Net Revenue (Comparable Base) (USD)], SAMEPERIODLASTYEAR ( dim_date[date] ) )
```

Net Revenue YoY % (Comparable Base) · format 0.0%

```
Net Revenue YoY % (Comparable Base) =  
DIVIDE ( [Net Revenue (Comparable Base) (USD)] - [Net Revenue (Comparable Base) PY (USD)], [Net Revenue (Comparable Base) PY (USD)] )
```

Expansion Contribution % · format 0.0%

```
Expansion Contribution % =  
DIVIDE ( [Net Revenue (Expansion) (USD)], [Net Revenue (USD)] )
```

Orders (Comparable Base) · format #,0

```
Orders (Comparable Base) =  
CALCULATE ( [Orders], KEEPFILTERS ( dim_market[is_comparable_base] = TRUE ) )
```

Net Revenue (Comparable, Dynamic) (USD) · format "\$"#,0

```
Net Revenue (Comparable, Dynamic) (USD) =  
VAR _pyStart = EDATE ( MIN ( dim_date[date] ), -12 )  
VAR _eligible = FILTER ( ALL ( dim_market ), dim_market[live_date] <= _pyStart )  
RETURN CALCULATE ( [Net Revenue (USD)], KEEPFILTERS ( _eligible ) )
```

04 Targets vs Plan

Net Revenue Target (USD) · format "\$"#,0

```
Net Revenue Target (USD) =  
CALCULATE ( SUM ( fact_target[target_value] ), fact_target[metric] = "Net Revenue" )
```

Net Revenue vs Target (USD) · format "\$"#,0

```
Net Revenue vs Target (USD) =  
[Net Revenue (USD)] - [Net Revenue Target (USD)]
```

Net Revenue vs Target % · format 0.0%

```
Net Revenue vs Target % =  
DIVIDE ( [Net Revenue (USD)], [Net Revenue Target (USD)] ) - 1
```

Orders Target · format #,0

```
Orders Target =  
CALCULATE ( SUM ( fact_target[target_value] ), fact_target[metric] = "Orders" )
```

Orders vs Target % · format 0.0%

```
Orders vs Target % =  
DIVIDE ( [Orders], [Orders Target] ) - 1
```

Gross Margin % Target · format 0.0%

```
Gross Margin % Target =  
CALCULATE ( AVERAGE ( fact_target[target_value] ), fact_target[metric] = "Gross Margin %" )
```

Gross Margin % vs Target (pp) · format #,0.0

```
Gross Margin % vs Target (pp) =  
( [Gross Margin %] - [Gross Margin % Target] ) * 100
```

Blended CAC Target · format "\$"#,0.00

```
Blended CAC Target =  
CALCULATE ( AVERAGE ( fact_target[target_value] ), fact_target[metric] = "Blended CAC" )
```

Blended CAC vs Target % · format 0.0%

```
Blended CAC vs Target % =  
DIVIDE ( [Blended CAC (USD)], [Blended CAC Target] ) - 1
```

New Customers Target · format #,0

```
New Customers Target =  
CALCULATE ( SUM ( fact_target[target_value] ), fact_target[metric] = "New Customers" )
```

New Customers vs Target % · format 0.0%

```
New Customers vs Target % =  
DIVIDE ( [New Customers], [New Customers Target] ) - 1
```

Budget Actual · format #,0.###

```
Budget Actual =  
SWITCH (   
    SELECTEDVALUE ( dim_metric[metric] ),  
    "Net Revenue", [Net Revenue (USD)],  
    "Orders", [Orders],  
    "New Customers", [New Customers],  
    "Gross Margin %", [Gross Margin %],  
    "Blended CAC", [Blended CAC (USD)],  
    BLANK ()  
 )
```

Budget Target · format #,0.###

```
Budget Target =  
VAR _m = SELECTEDVALUE ( dim_metric[metric] )  
RETURN  
    IF (   
        _m IN { "Gross Margin %", "Blended CAC" },  
        CALCULATE ( AVERAGE ( fact_target[target_value] ) ),  
        CALCULATE ( SUM ( fact_target[target_value] ) )  
    )
```

Budget Attainment % · format 0.0%

```
Budget Attainment % =  
VAR _a = [Budget Actual]  
VAR _t = [Budget Target]  
VAR _lowerBetter = SELECTEDVALUE ( dim_metric[metric] ) = "Blended CAC"  
RETURN DIVIDE ( IF ( _lowerBetter, _t - _a, _a - _t ), _t )
```

Budget Actual (fmt)

```
Budget Actual (fmt) =  
VAR _m = SELECTEDVALUE ( dim_metric[metric] )  
VAR _v = [Budget Actual]  
RETURN  
    SWITCH ( _m,  
        "Net Revenue", FORMAT ( _v / 1000000, "$#,0.0" ) & " M",  
        "Blended CAC", FORMAT ( _v, "$#,0.00" ),  
        "Gross Margin %", FORMAT ( _v, "0.0%" ),  
        FORMAT ( _v, "#,0" ) )
```

Budget Target (fmt)

```

Budget Target (fmt) =
VAR _m = SELECTEDVALUE ( dim_metric[metric] )
VAR _v = [Budget Target]
RETURN
    SWITCH ( _m,
        "Net Revenue", FORMAT ( _v / 1000000, "$#,0.0" ) & " M",
        "Blended CAC", FORMAT ( _v, "$#,0.00" ),
        "Gross Margin %", FORMAT ( _v, "0.0%" ),
        FORMAT ( _v, "#,0" ) )

```

Net Revenue Target (Category Alloc) (USD) · format "\$#,0

```

Net Revenue Target (Category Alloc) (USD) =
VAR _share = DIVIDE (
    [Net Revenue (USD)],
    CALCULATE ( [Net Revenue (USD)], REMOVEFILTERS ( dim_product ) )
)
RETURN CALCULATE ( [Net Revenue Target (USD)], REMOVEFILTERS ( dim_product ) ) * _share

```

Net Revenue vs Category Plan (USD) · format "\$#,0

```

Net Revenue vs Category Plan (USD) =
[Net Revenue (USD)] - [Net Revenue Target (Category Alloc) (USD)]

```

05 Reporting Currency

Net Revenue (Rpt) · format #,0

```

Net Revenue (Rpt) =
[Net Revenue (USD)] * [FX Rate to Reporting Currency]

```

Gross Profit (Rpt) · format #,0

```

Gross Profit (Rpt) =
[Gross Profit (USD)] * [FX Rate to Reporting Currency]

```

AOV (Rpt) · format #,0.00

```

AOV (Rpt) =
[Average Order Value] * [FX Rate to Reporting Currency]

```

Marketing Spend (Rpt) · format #,0

```

Marketing Spend (Rpt) =
[Marketing Spend (USD)] * [FX Rate to Reporting Currency]

```

Contribution Margin (Rpt) · format #,0

```

Contribution Margin (Rpt) =
[Contribution Margin (USD)] * [FX Rate to Reporting Currency]

```

06 Payments & Cash

Payment Fees (USD) · format "\$#,0

```

Payment Fees (USD) =
CALCULATE ( SUM ( fact_orders[payment_fee_usd] ), KEEPFILTERS ( fact_orders[is_cancelled] = FALSE ) )

```

Effective Fee Rate % · format 0.0%

```
Effective Fee Rate % =  
DIVIDE ( [Payment Fees (USD)], [Gross Revenue (USD)] )
```

Financing Cost Interest-Free (USD) · format "\$"#,0

```
Financing Cost Interest-Free (USD) =  
CALCULATE ( SUM ( fact_payment_schedule[financing_cost_usd] ), fact_payment_schedule[installment_plan] = "Interest-Free" )
```

Total Payment Cost (USD) · format "\$"#,0

```
Total Payment Cost (USD) =  
[Payment Fees (USD)] + [Financing Cost Interest-Free (USD)]
```

Installment Orders · format #,0

```
Installment Orders =  
CALCULATE ( DISTINCTCOUNT ( fact_orders[order_id] ), fact_orders[installments] > 1, KEEPFILTERS ( fact_orders[is_cancelled] = FALSE
```

Installment Mix % · format 0.0%

```
Installment Mix % =  
DIVIDE ( [Installment Orders], [Orders] )
```

Avg Installments (plans only) · format #,0.0

```
Avg Installments (plans only) =  
CALCULATE ( AVERAGE ( fact_orders[installments] ), fact_orders[installments] > 1, KEEPFILTERS ( fact_orders[is_cancelled] = FALSE )
```

Cash Received (USD) · format "\$"#,0

```
Cash Received (USD) =  
CALCULATE (   
    SUM ( fact_payment_schedule[net_amount_usd] ),  
    USERELATIONSHIP ( fact_payment_schedule[due_date_key], dim_date[date_key] )  
)
```

Revenue Booked (USD) · format "\$"#,0

```
Revenue Booked (USD) =  
[Gross Sales (USD)] + CALCULATE ( SUM ( fact_orders[tax_amount_usd] ) + SUM ( fact_orders[shipping_fee_usd] ), KEEPFILTERS ( fact_o
```

Revenue Booked (Cumulative) (USD) · format "\$"#,0

```
Revenue Booked (Cumulative) (USD) =  
CALCULATE (   
    [Revenue Booked (USD)],  
    FILTER ( ALL ( dim_date ), dim_date[date] <= MAX ( dim_date[date] ) )  
)
```

Cash Received (Cumulative) (USD) · format "\$"#,0

```
Cash Received (Cumulative) (USD) =  
CALCULATE (   
    SUM ( fact_payment_schedule[net_amount_usd] ),  
    USERELATIONSHIP ( fact_payment_schedule[due_date_key], dim_date[date_key] ),  
    FILTER ( ALL ( dim_date ), dim_date[date] <= MAX ( dim_date[date] ) )  
)
```

Booked vs Cash Gap (USD) · format "\$"#,0

```
Booked vs Cash Gap (USD) =  
[Revenue Booked (Cumulative) (USD)] - [Cash Received (Cumulative) (USD)]
```

Receivables Outstanding (USD) · format "\$"#,0

```
Receivables Outstanding (USD) =  
VAR _asOfKey = CALCULATE ( MAX ( dim_date[date_key] ), ALLSELECTED ( dim_date ) )  
VAR _booked =  
    CALCULATE ( SUM ( fact_payment_schedule[gross_amount_usd] ),  
        FILTER ( ALL ( fact_payment_schedule ), fact_payment_schedule[order_date_key] <= _asOfKey ),  
        REMOVEFILTERS ( dim_date ) )  
VAR _settled =  
    CALCULATE ( SUM ( fact_payment_schedule[gross_amount_usd] ),  
        FILTER ( ALL ( fact_payment_schedule ), fact_payment_schedule[due_date_key] <= _asOfKey ),  
        REMOVEFILTERS ( dim_date ) )  
RETURN _booked - _settled
```

DSO (days) · format #,0.0" d"

```
DSO (days) =  
DIVIDE (   
    SUMX ( fact_payment_schedule, fact_payment_schedule[gross_amount_usd] * fact_payment_schedule[settlement_lag_days] ),  
    SUM ( fact_payment_schedule[gross_amount_usd] )  
)
```

07 Price / Volume / Mix

Avg Selling Price · format "\$"#,0.00

```
Avg Selling Price =  
DIVIDE ( [Gross Revenue (USD)], [Units Sold] )
```

Avg Selling Price PY · format "\$"#,0.00

```
Avg Selling Price PY =  
CALCULATE ( [Avg Selling Price], SAMEPERIODLASTYEAR ( dim_date[date] ) )
```

Units Sold PY · format #,0

```
Units Sold PY =  
CALCULATE ( [Units Sold], SAMEPERIODLASTYEAR ( dim_date[date] ) )
```

PVM Price Effect (USD) · format "\$"#,0

```
PVM Price Effect (USD) =  
SUMX (   
    VALUES ( dim_product[category] ),  
    VAR _pxCY = [Avg Selling Price]  
    VAR _pxPY = [Avg Selling Price PY]  
    VAR _qtyPY = [Units Sold PY]  
    RETURN ( _pxCY - _pxPY ) * _qtyPY  
)
```

PVM Volume Effect (USD) · format "\$"#,0

```
PVM Volume Effect (USD) =  
VAR _totQtyPY = [Units Sold PY]  
VAR _totRevPY = [Gross Revenue (USD) PY]  
VAR _growth = DIVIDE ( [Units Sold] - _totQtyPY, _totQtyPY )  
RETURN _totRevPY * _growth
```

Gross Revenue (USD) PY · format "\$"#,0

```
Gross Revenue (USD) PY =  
CALCULATE ( [Gross Revenue (USD)], SAMEPERIODLASTYEAR ( dim_date[date] ) )
```

PVM Mix Effect (USD) · format "\$"#,0

```
PVM Mix Effect (USD) =  
[Gross Revenue (USD)] - [Gross Revenue (USD) PY] - [PVM Price Effect (USD)] - [PVM Volume Effect (USD)]
```

08 Marketing & Acquisition

Marketing Spend (USD) · format "\$"#,0

```
Marketing Spend (USD) =  
SUM ( fact_marketing_spend[cost_usd] )
```

Impressions · format #,0

```
Impressions =  
SUM ( fact_marketing_spend[impressions] )
```

Clicks · format #,0

```
Clicks =  
SUM ( fact_marketing_spend[clicks] )
```

CTR % · format 0.0%

```
CTR % =  
DIVIDE ( [Clicks], [Impressions] )
```

CPC (USD) · format "\$"#,0.00

```
CPC (USD) =  
DIVIDE ( [Marketing Spend (USD)], [Clicks] )
```

New Customers · format #,0

```
New Customers =  
VAR _newCust =  
    FILTER (  
        VALUES ( dim_customer[customer_key] ),  
        CALCULATE ( MIN ( fact_orders[order_date_key] ) ) =  
        CALCULATE ( MIN ( fact_orders[order_date_key] ), REMOVEFILTERS ( dim_date ) )  
    )  
RETURN  
    CALCULATE (  
        DISTINCTCOUNT ( fact_orders[customer_key] ),  
        KEEPFILTERS ( _newCust ),  
        KEEPFILTERS ( fact_orders[is_cancelled] = FALSE )  
    )
```

Returning Customers · format #,0

```
Returning Customers =  
[Customers with Orders] - [New Customers]
```

New Customer Revenue (USD) · format "\$"#,0

```
New Customer Revenue (USD) =  
VAR _newCust =  
    FILTER (
```

```
VALUES ( dim_customer[customer_key] ),
CALCULATE ( MIN ( fact_orders[order_date_key] ) ) =
CALCULATE ( MIN ( fact_orders[order_date_key] ), REMOVEFILTERS ( dim_date ) )
)
RETURN CALCULATE ( [Net Revenue (USD)], KEEPFILTERS ( _newCust ) )
```

Returning Customer Revenue (USD) · format "\$"#,0

```
Returning Customer Revenue (USD) =
[Net Revenue (USD)] - [New Customer Revenue (USD)]
```

New Revenue % · format 0.0%

```
New Revenue % =
DIVIDE ( [New Customer Revenue (USD)], [Net Revenue (USD)] )
```

Blended CAC (USD) · format "\$"#,0.00

```
Blended CAC (USD) =
DIVIDE ( [Marketing Spend (USD)], [New Customers] )
```

% New Customers via Paid · format 0.0%

```
% New Customers via Paid =
DIVIDE (
    CALCULATE ( [New Customers (acq month)],
        dim_customer[acquisition_channel] IN { "Paid Search", "Paid Social", "Display", "Affiliate" } ),
    [New Customers (acq month)]
)
```

ROAS · format #,0.0"x"

```
ROAS =
DIVIDE ( [Net Revenue (USD)], [Marketing Spend (USD)] )
```

MER · format #,0.0"x"

```
MER =
DIVIDE ( [Net Revenue (USD)], [Marketing Spend (USD)] )
```

Marketing Spend PY (USD) · format "\$"#,0

```
Marketing Spend PY (USD) =
CALCULATE ( [Marketing Spend (USD)], SAMEPERIODLASTYEAR ( dim_date[date] ) )
```

Marketing Spend YoY % · format 0.0%

```
Marketing Spend YoY % =
DIVIDE ( [Marketing Spend (USD)] - [Marketing Spend PY (USD)], [Marketing Spend PY (USD)] )
```

09 Website & Funnel

Sessions · format #,0

```
Sessions =
SUM ( fact_web_traffic_daily[sessions] )
```

Users · format #,0

```
Users =
```

```
SUM ( fact_web_traffic_daily[users] )
```

Pageviews · format #,0

```
Pageviews =  
SUM ( fact_web_traffic_daily[pageviews] )
```

Bounce Rate % · format 0.0%

```
Bounce Rate % =  
DIVIDE ( SUM ( fact_web_traffic_daily[bounces] ), [Sessions] )
```

Pages per Session · format #,0.0

```
Pages per Session =  
DIVIDE ( [Pageviews], [Sessions] )
```

Add-to-Cart Rate % · format 0.0%

```
Add-to-Cart Rate % =  
DIVIDE ( SUM ( fact_web_traffic_daily[add_to_carts] ), [Sessions] )
```

Add-to-Carts · format #,0

```
Add-to-Carts =  
SUM ( fact_web_traffic_daily[add_to_carts] )
```

Carts Created · format #,0

```
Carts Created =  
SUM ( fact_web_traffic_daily[carts_created] )
```

Cart Abandonment Rate % · format 0.0%

```
Cart Abandonment Rate % =  
1 - DIVIDE ( [Orders], [Carts Created] )
```

Conversion Rate % · format 0.0%

```
Conversion Rate % =  
DIVIDE ( [Orders], [Sessions] )
```

Revenue per Session (USD) · format "\$"#,0.00

```
Revenue per Session (USD) =  
DIVIDE ( [Net Revenue (USD)], [Sessions] )
```

Funnel Impressions · format #,0

```
Funnel Impressions =  
[Impressions]
```

Funnel Clicks · format #,0

```
Funnel Clicks =  
[Clicks]
```

Funnel Sessions · format #,0

```
Funnel Sessions =  
[Sessions]
```

Funnel Carts · **format** #,0

```
Funnel Carts =  
[Carts Created]
```

Funnel Orders · **format** #,0

```
Funnel Orders =  
[Orders]
```

Funnel Value · **format** #,0

```
Funnel Value =  
SWITCH (   
    SELECTEDVALUE ( 'Funnel Stage'[Stage] ),  
    "Sessions", [Sessions],  
    "Add-to-Cart", [Add-to-Carts],  
    "Cart", [Carts Created],  
    "Orders", [Orders],  
    BLANK ()  
 )
```

10 Logistics & Fulfillment

Orders Delivered · **format** #,0

```
Orders Delivered =  
CALCULATE ( DISTINCTCOUNT ( fact_orders[order_id] ), fact_orders[is_delivered] = TRUE )
```

Avg Delivery Days · **format** #,0.0" d"

```
Avg Delivery Days =  
CALCULATE ( AVERAGE ( fact_orders[delivery_days] ), fact_orders[is_delivered] = TRUE )
```

Avg Ship Days · **format** #,0.0" d"

```
Avg Ship Days =  
AVERAGE ( fact_orders[ship_days] )
```

On-Time Delivery % · **format** 0.0%

```
On-Time Delivery % =  
DIVIDE ( CALCULATE ( DISTINCTCOUNT ( fact_orders[order_id] ), fact_orders[is_on_time] = TRUE ), [Orders Delivered] )
```

Late Deliveries · **format** #,0

```
Late Deliveries =  
CALCULATE ( DISTINCTCOUNT ( fact_orders[order_id] ), fact_orders[is_late] = TRUE, fact_orders[is_delivered] = TRUE )
```

Perfect Order Rate % · **format** 0.0%

```
Perfect Order Rate % =  
DIVIDE ( CALCULATE ( DISTINCTCOUNT ( fact_orders[order_id] ), fact_orders[is_perfect_order] = TRUE ), [Orders] )
```

Shipping Cost (USD) · **format** "\$"#,0

```
Shipping Cost (USD) =  
CALCULATE ( SUM ( fact_orders[shipping_cost_usd] ), KEEPFILTERS ( fact_orders[is_cancelled] = FALSE ) )
```

Shipping Cost per Order (USD) · format "\$"#,0.00

```
Shipping Cost per Order (USD) =  
DIVIDE ( [Shipping Cost (USD)], [Orders] )
```

Shipping Fee Revenue (USD) · format "\$"#,0

```
Shipping Fee Revenue (USD) =  
CALCULATE ( SUM ( fact_orders[shipping_fee_usd] ), KEEPFILTERS ( fact_orders[is_cancelled] = FALSE ) )
```

Shipping Cost Recovery % · format 0.0%

```
Shipping Cost Recovery % =  
DIVIDE ( [Shipping Fee Revenue (USD)], [Shipping Cost (USD)] )
```

11 Working Capital & Inventory

Inventory Value (USD) · format "\$"#,0

```
Inventory Value (USD) =  
VAR _lastSnap = CALCULATE ( MAX ( fact_inventory_snapshot[snapshot_date_key] ), ALLSELECTED ( dim_date ) )  
RETURN CALCULATE ( SUM ( fact_inventory_snapshot[inventory_value_usd] ), fact_inventory_snapshot[snapshot_date_key] = _lastSnap )
```

Inventory Value (Month-End) (USD) · format "\$"#,0

```
Inventory Value (Month-End) (USD) =  
VAR _snap = MAX ( fact_inventory_snapshot[snapshot_date_key] )  
RETURN CALCULATE ( SUM ( fact_inventory_snapshot[inventory_value_usd] ),  
    fact_inventory_snapshot[snapshot_date_key] = _snap )
```

Weeks of Cover (Month-End) · format #,0.0

```
Weeks of Cover (Month-End) =  
VAR _snap = MAX ( fact_inventory_snapshot[snapshot_date_key] )  
RETURN CALCULATE ( DIVIDE ( SUM ( fact_inventory_snapshot[units_on_hand] ),  
    SUM ( fact_inventory_snapshot[avg_weekly_units_sold] ) ),  
    fact_inventory_snapshot[snapshot_date_key] = _snap )
```

Inventory in Transit (USD) · format "\$"#,0

```
Inventory in Transit (USD) =  
VAR _lastSnap = CALCULATE ( MAX ( fact_inventory_snapshot[snapshot_date_key] ), ALLSELECTED ( dim_date ) )  
RETURN CALCULATE ( SUMX ( fact_inventory_snapshot, fact_inventory_snapshot[units_in_transit] * fact_inventory_snapshot[moving_avg_c
```

Avg Inventory Value (USD) · format "\$"#,0

```
Avg Inventory Value (USD) =  
AVERAGEX ( VALUES ( fact_inventory_snapshot[snapshot_date_key] ), CALCULATE ( SUM ( fact_inventory_snapshot[inventory_value_usd] ) )
```

COGS Trailing 12M (USD) · format "\$"#,0

```
COGS Trailing 12M (USD) =  
CALCULATE ( [COGS (USD)], DATESINPERIOD ( dim_date[date], MAX ( dim_date[date] ), -12, MONTH ) )
```

Inventory Turnover · format `#,0.0"x"`

```
Inventory Turnover =  
DIVIDE ( [COGS Trailing 12M (USD)], [Avg Inventory Value (USD)] )
```

DIO (days) · format `#,0.0" d"`

```
DIO (days) =  
DIVIDE ( [Avg Inventory Value (USD)] * 365, [COGS Trailing 12M (USD)] )
```

Accounts Payable Open (USD) · format `"$"#,0`

```
Accounts Payable Open (USD) =  
VAR _asOfKey = CALCULATE ( MAX ( dim_date[date_key] ), ALLSELECTED ( dim_date ) )  
RETURN  
CALCULATE (   
    SUM ( fact_purchase_orders[po_value_usd] ),  
    FILTER (   
        ALL ( fact_purchase_orders ),  
        fact_purchase_orders[actual_receipt_date_key] <= _asOfKey  
        && fact_purchase_orders[supplier_paid_date_key] > _asOfKey  
    ),  
    REMOVEFILTERS ( dim_date )  
)
```

DPO (days) · format `#,0.0" d"`

```
DPO (days) =  
DIVIDE ( [Accounts Payable Open (USD)] * 365, [COGS Trailing 12M (USD)] )
```

Accounts Receivable Open (USD) · format `"$"#,0`

```
Accounts Receivable Open (USD) =  
[Receivables Outstanding (USD)]
```

Cash Conversion Cycle (days) · format `#,0.0" d"`

```
Cash Conversion Cycle (days) =  
[DIO (days)] + [DSO (days)] - [DPO (days)]
```

Weeks of Cover · format `#,0.0`

```
Weeks of Cover =  
VAR _lastSnap = CALCULATE ( MAX ( fact_inventory_snapshot[snapshot_date_key] ), ALLSELECTED ( dim_date ) )  
RETURN CALCULATE (   
    DIVIDE ( SUM ( fact_inventory_snapshot[units_on_hand] ), SUM ( fact_inventory_snapshot[avg_weekly_units_sold] ) ),  
    fact_inventory_snapshot[snapshot_date_key] = _lastSnap )
```

Supplier Spend (USD) · format `"$"#,0`

```
Supplier Spend (USD) =  
CALCULATE ( SUM ( fact_purchase_orders[po_value_usd] ),  
    USERELATIONSHIP ( fact_purchase_orders[actual_receipt_date_key], dim_date[date_key] ) )
```

Working Capital Tied Up (USD) · format `"$"#,0`

```
Working Capital Tied Up (USD) =  
[Inventory Value (USD)] + [Accounts Receivable Open (USD)] - [Accounts Payable Open (USD)]
```

12 CRM & Retention

Customers with Orders · format `#,0`

```
Customers with Orders =  
CALCULATE ( DISTINCTCOUNT ( fact_orders[customer_key] ), KEEPFILTERS ( fact_orders[is_cancelled] = FALSE ) )
```

Active Customers (12M) · format #,0

```
Active Customers (12M) =  
CALCULATE ( DISTINCTCOUNT ( fact_orders[customer_key] ), DATESINPERIOD ( dim_date[date], MAX ( dim_date[date] ), -12, MONTH ), KEEP
```

Avg Orders per Customer · format #,0.0

```
Avg Orders per Customer =  
DIVIDE ( [Orders], [Customers with Orders] )
```

Repeat Purchase Rate % · format 0.0%

```
Repeat Purchase Rate % =  
VAR _cust = CALCULATETABLE ( VALUES ( fact_orders[customer_key] ), fact_orders[is_cancelled] = FALSE )  
VAR _repeat = FILTER ( _cust, CALCULATE ( DISTINCTCOUNT ( fact_orders[order_id] ) ) > 1 )  
RETURN DIVIDE ( COUNTROWS ( _repeat ), COUNTROWS ( _cust ) )
```

CLV Gross Profit (USD) · format "\$"#,0.00

```
CLV Gross Profit (USD) =  
DIVIDE ( [Gross Profit (USD)], [Customers with Orders] )
```

New Customers (acq month) · format #,0

```
New Customers (acq month) =  
DISTINCTCOUNT ( dim_customer[customer_id] )
```

Repeat Customers (lifetime) · format #,0

```
Repeat Customers (lifetime) =  
CALCULATE ( DISTINCTCOUNT ( dim_customer[customer_id] ), dim_customer[is_repeat_customer] = TRUE )
```

Churned Customers · format #,0

```
Churned Customers =  
CALCULATE ( DISTINCTCOUNT ( dim_customer[customer_id] ), dim_customer[customer_status] = "Churned" )
```

Gold Tier Revenue % · format 0.0%

```
Gold Tier Revenue % =  
DIVIDE ( CALCULATE ( [Net Revenue (USD)], dim_customer[loyalty_tier] = "Gold" ), [Net Revenue (USD)] )
```

13 Support & Returns

Support Tickets · format #,0

```
Support Tickets =  
DISTINCTCOUNT ( fact_support_tickets[ticket_id] )
```

Avg Resolution Hours · format #,0.0

```
Avg Resolution Hours =  
AVERAGE ( fact_support_tickets[resolution_hours] )
```

First Contact Resolution % · format 0.0%

First Contact Resolution % =
DIVIDE (CALCULATE (DISTINCTCOUNT (fact_support_tickets[ticket_id]), fact_support_tickets[first_contact_resolution] = TRUE), [S

Avg CSAT · format #,0.00

Avg CSAT =
AVERAGE (fact_support_tickets[csat])

Tickets per 1000 Orders · format #,0.0

Tickets per 1000 Orders =
DIVIDE ([Support Tickets], [Orders]) * 1000

Returned Units · format #,0

Returned Units =
SUM (fact_returns[returned_qty])

Return Value (USD) · format "\$"#,0

Return Value (USD) =
SUM (fact_returns[refund_amount_usd])

Return Rate % · format 0.0%

Return Rate % =
DIVIDE (CALCULATE (SUM (fact_order_lines[returned_qty]), KEEPFILTERS (fact_order_lines[order_status] <> "cancelled")), [Unit

Restock Rate % · format 0.0%

Restock Rate % =
DIVIDE (CALCULATE (SUM (fact_returns[returned_qty]), fact_returns[restocked] = TRUE), [Returned Units])

07 — Report Guide

The Power BI report is a **thick .pbip project**: a local semantic model (TMDL, generated from the curated Parquet schemas) plus a PBIR report. Open `powerbi/VoltEdge_Electronics.pbip` in Power BI Desktop; it imports `data/curated/*.parquet` through the `pDataFolder` parameter and applies the 145 measures.

- **Model:** `etl/gen_semantic_model.py` → `powerbi/VoltEdge_Electronics.SemanticModel/`
- **Measures:** `etl/pbi_measures.py` → `_Measures` table (documented in [06_dax_measures.md](#))
- **Report layout:** `etl/gen_report_visuals.py` → per-page `--from-json` bundles
- **Theme:** `powerbi/theme/VoltEdge.json`
- **One command rebuilds everything:** `cd powerbi && bash build_report.sh`

Design identity (v2 — deep blue)

Choice	Decision
Tone	Corporate — one accent family, muted everything else, quiet gridlines
Palette	Deep blue <code>#123E6E</code> = current period / focus · navy <code>#0E2A4E</code> = header · amber <code>#D99311</code> = prior year / comparison · mid-blue <code>#2E70B0</code> for a second series · the rest neutral grays
Signature	Every page: a navy logo bar (<code>#0E2A4E</code> , 56 px) with the VoltEdge mark top-left and a <code>#2E70B0</code> accent rule beneath, the page title + two slicers (Currency, Market) on the line below, then one row of 6 KPI cards, then two content rows on the 3-30-300 detail gradient
Logo	<code>powerbi/assets/voltedge-logo-*.png</code> , generated by <code>etl/gen_logo.py</code> (a lightning-bolt mark + wordmark; light version for the navy bar, dark version for light backgrounds)
Surfaces	Page ground <code>#EAEFF6</code> ; visuals are white cards, 12 px corner radius, 1 px <code>#DCE4EF</code> border, no drop shadow
Type	Segoe UI / Segoe UI Semibold; KPI value 32 pt deep blue, no duplicate label, titles 13 pt semibold, axis/label 10 pt <code>#64707F</code>
Charts	Axis titles off, category gridlines off, value gridlines <code>#EAEFF6</code> , legends top-center no title, line stroke 2 px no markers; time axes sorted oldest→newest (model <code>sortByColumn</code> + explicit visual sort)
Tables	No vertical gridlines, no row banding, header on <code>#EAEFF6</code> — whitespace separates rows

The theme owns all of the above; visuals carry almost no local overrides, so a rebrand is a one-file change. `pbir theme validate` passes.

If the curated data lives elsewhere, change the one parameter `pDataFolder` (Transform data → Edit parameters) and refresh.

Model at a glance

- **28 data tables** (16 dims + 12 facts) + `_Measures` + two disconnected helper tables (`Reporting_Currency` , `Funnel_Stage`).
- **65 relationships**, all single-direction many-to-one from fact to dimension. `dim_date` joins each fact on its primary date key (active); secondary date roles (ship, delivery, due, receipt, paid...) are **inactive** relationships activated inside specific measures with `USERELATIONSHIP` .
- **Reporting-currency selector** — the disconnected `Reporting_Currency` slicer drives `[FX Rate to Reporting Currency]` ; the (Rpt) measures multiply the USD base by it. Default is USD.
- **Comparable Base** — `dim_market[is_comparable_base]` (US) gives the like-for-like cut; `[Net Revenue (Comparable, Dynamic)]` recomputes the eligible market set from the visible date range.

Reading the numbers

- **Contribution Margin** = Gross Profit – Marketing – Shipping Cost – Payment Fees – merchant financing. It runs **near break-even in normal months (~–2 %)** and dips hard in the Q4 promo pushes (~–13 %), where discount depth cuts gross margin to ~7 % while marketing stays high. Blended ≈ –5 %. The **CM bridge waterfall** on the Executive Summary shows exactly where the money goes; the monthly CM % trend shows the seasonality. Returns are netted correctly — the COGS of restocked returns is credited back (a ~\$2 M swing versus naively subtracting only the refund).
- **Revenue Booked vs Cash Received** — Booked is recognised at order date; Cash is when each settlement tranche is actually paid. The gap (~5–7 % of cumulative revenue) is the **receivables float**: card settlement lag, an 8 % processor rolling reserve released at +90 days, and Brazilian instalments spread over months. Shown as **cumulative** lines on Sales Performance so the widening wedge is visible.
- **Acquisition is not all paid** — ~62 % of new customers come through paid channels, ~38 % organic / direct / email / referral. The *New customers by acquisition channel* chart on Marketing makes that explicit; Blended CAC deliberately divides total spend by *all* new customers.

Pages

1280×860 canvas, 24 px margins, 16 px gutters. Every page: **header image** (navy bar: bolt + VOLTEDGE + page title) · **slicer row** (Analysis period date-range, Currency & Market as horizontal tiles) · **6 KPI tiles** (`kpi` visual with a plan Goal + trend sparkline where a target exists, `card` otherwise) · **two**

content rows on the 3-30-300 detail gradient.

Every "big number + ratio" pair is a **combo chart** — magnitude as bars, ratio as a line on a secondary axis (sessions/CR, delivery-days/on-time, spend/ROAS, inventory-value/ weeks-of-cover, ...). Time axes are sorted oldest→newest.

1 • Executive Summary — are we hitting plan, is growth real, where does the money go?

- **KPI tiles:** Net Revenue, Gross Margin %, Orders (all kpi w/ plan Goal + sparkline) · Contribution Margin (card) · New Customers, Blended CAC (kpi w/ Goal).
- **Net revenue by month — CY vs PY** (line).
- **Budget scorecard** (table): each planned metric — actual, target, variance %.
- **Contribution margin bridge** (waterfall): Gross Profit → –Marketing → –Shipping → –Payment Fees → –Financing → CM. Answers "why is CM where it is?".
- **Net revenue by market — CY vs PY** (column): the expansion step-ups.

2 • Sales Performance

- KPI tiles: Net Revenue, **Orders**, AOV, Gross Margin %, Discount Rate %, Units/Order.
- **Net revenue (bars) & discount rate (line) by month** (combo).
- **Net revenue by category** (bar).
- **YoY revenue change by category** (waterfall) — PVM detail in the 07 ... measures.
- **Cumulative revenue booked vs cash received** (line) — the receivables float.

3 • Marketing & Acquisition

- KPI tiles: Marketing Spend, Blended CAC, MER, New Customers, **% New via Paid**, ROAS.
- **On-site funnel:** Sessions → Add-to-Cart → Cart → Orders.
- **Spend (bars) & blended CAC (line) by month** (combo).
- **New customers by acquisition channel** (bar) — the paid vs organic split.
- **Spend (bars) & ROAS (line) by channel** (combo) · **Campaign performance** (table).

4 • Website & Digital

- KPI tiles: Sessions, Conversion Rate %, Bounce Rate %, Add-to-Cart %, Cart Abandonment %, Revenue / Session.
- **Sessions (bars) & conversion rate (line) by month** (combo) · **Sessions by device** (donut).
- **Revenue/session (bars) & conversion rate (line) by channel** (combo) · **Sessions (bars) & conversion rate (line) by market** (combo).

5 • Logistics & Fulfillment

- KPI tiles: On-Time Delivery %, Avg Delivery Days, Perfect Order %, Ship Cost/Order, Cost Recovery %, Cash Conversion Cycle.
- **Avg delivery days (bars) & on-time % (line) by month** (combo) — monthly logistics stress and carrier-crisis months are now visible.
- **Carrier scorecard** (combo: days vs on-time %).
- **Working capital — DIO / DSO / DPO / CCC by month** (column).
- **Company cash flow** (line): settlements in vs supplier + marketing out.

6 • CRM & Customer

- KPI tiles: Customers with Orders, Repeat Rate %, Avg Orders/Customer, CLV (Gross Profit), Churned Customers, Gold-Tier Revenue %.
- **New vs returning customer revenue by month** (line) — the two lines weave and cross.
- **Customers by status** (bar).
- **Customers acquired by cohort month & channel** (stacked column).
- **Returns by reason** (table: units, value, restock %).

7 • Product & Inventory

- KPI tiles: Inventory Value, Inventory Turnover, Weeks of Cover, Return Rate %, In Transit, Supplier Spend.
- **Gross margin % by category** (bar) · **Return rate % by category** (bar) · **SKUs — revenue vs margin** (scatter).
- **Inventory value (bars) & weeks of cover (line) by month** (combo) — month-end measures, so every month renders.
- **Slow movers** (table).

Known polish backlog (structural report is valid; pbir validate --all passes, no overlaps)

Item	Note
KPI cards / slicers flagged VISUAL_UNDSIZED	192×84 px cards and 210×44 slicers are deliberately compact for the 6-up band and header strip.
TOO_MANY_FIELDS ×2	The DIO/DSO/DPO/CCC column chart carries 4 series by design.
No alt text on charts yet	Add a pass of pbir set "...general.altText" before shipping.

Item	Note
KPI cards are plain <code>card</code> (no target/trend)	A <code>kpi</code> visual with target + trendline would be stronger on the hero metrics — model has the target measures.
Sub-topics dropped for density	Sales geo map, per-method installment mix, and a cohort-retention heatmap are candidates for a Phase-2 detail pass.
Rendering not yet verified in Desktop	<code>pbir</code> cannot drive Desktop here; some theme properties may need adjusting once seen.

Verifying in Power BI Desktop

1. Open `powerbi/VoltEdge Electronics.pbip`.
2. It should load the model and import the Parquet folder. If it asks, point `pDataFolder` at `.../VoltEdge-Electronics-Analytics/data/curated`.
3. Refresh. Check the model view for the star schema and inactive date relationships.
4. Page through the 7 report pages; confirm measures resolve (no `#ERROR` cards).
5. Report any measure or visual that misbehaves — that is the next iteration.

08 — Data Quality

Data quality is handled in two places and reported in one.

Where issues are handled

Stage	Role
01_generate_raw.py	Injects realistic defects into the raw layer (see the <code>DQ</code> block in <code>config.py</code> and the table in 02_data_architecture.md).
02_clean_stage.py	Fixes them — parsing, typing, de-duplication, encoding detection, category standardization, identity resolution, reconciliation — and counts every fix into a Markdown fragment.
dq_checks.py	Gates the curated layer with HARD / SOFT assertions and assembles the full report.

The report

`dq_checks.py` writes [../data/quality/dq_report.md](#) on every run. It contains:

- Curated-layer gate** — every structural and business assertion with pass/fail: - no null surrogate keys in fact tables; - referential integrity for every fact → dimension key (unresolved keys land on the `-1 Unknown` member, never dropped); - every fact date present in `dim_date`; - accounting identities: `net = gross - discount` per line; order lines roll up to the order header; the payment schedule sums back to the order total; - invariants: `comparable base = US only`, `FX USD = 1.0`, no non-positive rates; - sanity bands: order count, AOV, blended gross margin, share of below-cost lines. A **HARD** failure exits non-zero and halts `run_pipeline.py`.
- Stage 02 cleaning log** — counts for each fix applied (categories normalized, duplicate rows dropped, e-mails missing, encoding detected, PSP rows unmatched, snapshot-vs-ledger variance, negative sessions corrected, ...).

Known, accepted limitations

Item	Why it's acceptable
Web analytics only covers market-channel-device cells that had ≥ 1 order that day	Phase 1 uses web data only for the acquisition funnel; a zero-conversion backfill is a Phase 2 task.
Inventory is modelled at SKU × market-primary-warehouse grain	Order fulfilment can still split across a market's FCs for carrier analysis; inventory accounting rolls to the primary FC.
Affiliate cost is a commission on affiliate-channel orders; attribution is last-touch at channel level	Realistic ad-platform attribution is explicitly out of scope (see 01 , §11).
FX uses a synthetic mean-reverting series, not historical rates	The point is to exercise multi-currency modelling, not to reproduce real markets.

VoltEdge Electronics — Report Structure

Document status: built to match the numbered `docs/` convention used by the other four portfolio projects. Full design rationale, palette, and page-by-page detail already live in [07_report_guide.md](#) — this doc is the compact index of that same report, table-first. One command rebuilds the report:

```
cd powerbi && bash build_report.sh
```

Pipeline: `gen_logo.py` → `gen_semantic_model.py` (+ `pbi_measures.py`) → `gen_dax_doc.py` → `gen_report_visuals.py` → `build_report.sh` (drives `pbir`). Output: `powerbi/VoltEdge_Electronics.pbip` + `.Report` + `.SemanticModel`. `pbir validate --all` passes (see the known polish backlog in `07_report_guide.md`).

First open in Power BI Desktop: Transform data → set `pDataFolder` to `data/curated` → Close & Apply · View ▸ Themes ▸ Browse → `powerbi/theme/VoltEdge.json`.

Semantic model

28 data tables (16 dims + 12 facts) + `_Measures` (167 measures in 14 folders) + two disconnected helper tables (`Reporting_Currency` , `Funnel_Stage`). 65 relationships, single-direction many-to-one fact → dimension; secondary date roles (ship, delivery, due, receipt, paid...) are inactive, activated per-measure with `USERELATIONSHIP`. See `docs/05_data_model.md` for the full bus matrix.

The seven pages

Every page: navy logo bar + accent rule (header image) · Currency + Market slicer row · Analysis-period date slicer · 6 KPI tiles · two content rows on the 3-30-300 gradient.

#	Page	Question	KPI row (6 cards)	Content
1	Executive Summary	Are we hitting plan, is growth real, where does the money go?	Net Revenue · Gross Margin % · Orders · Contribution Margin · New Customers · Blended CAC	Net revenue by month CY vs PY (line) · budget scorecard (table) · contribution-margin bridge (waterfall) · net revenue by market CY vs PY (column)
2	Sales Performance	What is driving the top line — volume, price, or mix?	Net Revenue · Orders · AOV · Gross Margin % · Discount Rate % · Units/Order	Net revenue & discount rate by month (combo) · net revenue by category (bar) · YoY revenue change by category (waterfall, price/volume/mix) · cumulative revenue booked vs cash received (line)
3	Marketing & Acquisition	Is paid spend efficient, and how much growth is actually paid-for?	Marketing Spend · Blended CAC · MER · New Customers · % New via Paid · ROAS	On-site funnel (sessions → cart → orders) · spend & blended CAC by month (combo) · new customers by acquisition channel (bar) · spend & ROAS by channel (combo) · campaign performance (table)
4	Website & Digital	Where does the on-site funnel leak?	Sessions · Conversion Rate % · Bounce Rate % · Add-to-Cart % · Cart Abandonment % · Revenue/Session	Sessions & conversion rate by month (combo) · sessions by device (donut) · revenue/session & conversion by channel (combo) · sessions & conversion by market (combo)
5	Logistics & Fulfillment	Are we delivering on time, and where is cash trapped?	On-Time Delivery % · Avg Delivery Days · Perfect Order % · Ship Cost/Order · Cost Recovery % · Cash Conversion Cycle	Avg delivery days & on-time % by month (combo) · carrier scorecard (combo) · working capital — DIO/DSO/DPO/CCC by month (column) · company cash flow (line)
6	CRM & Customer	Is the customer base retaining, and who churns?	Customers with Orders · Repeat Rate % · Avg Orders/Customer · CLV (Gross Profit) · Churned Customers · Gold-Tier Revenue %	New vs returning customer revenue by month (line) · customers by status (bar) · customers acquired by cohort month & channel (stacked column) · returns by reason (table)
7	Product & Inventory	Which SKUs and categories drive margin, and where does inventory sit?	Inventory Value · Inventory Turnover · Weeks of Cover · Return Rate % · In Transit · Supplier Spend	Gross margin % by category (bar) · return rate % by category (bar) · SKUs — revenue vs margin (scatter) · inventory value & weeks of cover by month (combo) · slow movers (table)

Every "big number + ratio" pair is a combo chart — magnitude as bars, ratio as a line on a secondary axis. Time axes are sorted oldest→newest.

Known lite-scope gaps (not polished)

See `07_report_guide.md` 's known polish backlog for the full list — undersized KPI cards/slicers (deliberately compact), two `TOO_MANY_FIELDS` warnings on the working-capital column chart, no alt text yet, plain `card` visuals on the hero KPIs instead of `kpi` with target + trendline, and rendering not yet verified live in Desktop.

Walkthrough — What Was Built, Step by Step

The running narrative of the project: each entry records what was done, why, and where to look. Newest work at the bottom.

Step 0 — Scoping (done)

Decisions locked with the stakeholder:

Decision	Choice
Deliverable language	English
Power BI environment	Power BI Desktop available; deliver as <code>.pbip</code> (PBIR) + report
Data scale	Robust: ~3 years, ~30k customers, ~150k orders, ~300 SKUs
Data-engineering depth	Raw (messy, multi-source) + Python ETL + architecture diagram
Market scope	Multi-country / multi-currency: US, UK, DE, BR
Curated storage	CSV + Parquet (no Excel workbook)
Phase 1 scope	Foundation + 3 pages: Executive Summary, Sales Performance, Marketing & Acquisition

Step 1 — Project scaffold (done)

Created the folder structure (`docs/` , `etl/` , `data/{raw,staging,curated,quality}/` , `powerbi/` , `assets/`) and the top-level [README.md](#) .

Step 2 — Business context & KPI framework (done, awaiting review)

Wrote [01_business_context_kpis.md](#) : company profile, market expansion timeline, product catalog and margins, business model & financial definitions, embedded seasonality, stakeholder personas, and the full KPI framework (definitions + benchmarks) across Sales, Marketing, Web, Logistics and CRM. (Assumptions later resolved in §12.)

Step 3 — Data architecture (done, awaiting review)

Wrote [02_data_architecture.md](#) : medallion design principles, layer-by-layer spec, source-system inventory (10 simulated systems), the catalogue of data-quality issues to inject, tech stack, run instructions, and naming conventions. Includes the Mermaid architecture diagram.

Step 3a — Scope revision after stakeholder feedback (done)

Stakeholder adjustments folded into `01` and `02` :

- **Extract window shortened to 24 months** (2024-07-01 → 2026-06-30) for a clean YoY; expansion dates moved *inside* the window (UK+DE 2025-01, BR 2025-07) so **Total vs Comparable Base (US like-for-like)** analysis is possible. New `01` §5.
- **Payments deepened:** `dim_payment_method` with MDR by scheme, fixed fees, settlement lag; `fact_orders` + `fact_payment_schedule` (instalment tranches); interest-free merchant-funded vs customer-funded plans. New `01` §3.
- **Procurement & inventory added:** `dim_supplier` (Net 30/45/60 terms, lead time), `fact_purchase_orders` , `fact_inventory_movement` ledger → derived `fact_inventory_snapshot` ; moving-average COGS; DIO/DSO/DPO/**CCC** and a company cash-flow timeline. New `01` §4.
- Source systems grew from 10 to 12 (added PSP and Procurement). `02` §2–4 updated, Mermaid diagram redrawn.
- All open assumptions resolved — `01` §12.

Step 4 — ETL pipeline built and run (done)

`etl/` now holds the full four-stage pipeline (`config.py` , `utils.py` , `01_generate_raw.py` , `02_clean_stage.py` , `03_build_curated.py` , `dq_checks.py` , `run_pipeline.py` , `gen_data_dictionary.py`). Seeded, reproducible, ~10 min end to end.

- **Raw** — 19 files / 12 simulated source systems with injected mess (category spellings ×3, money strings, 5 date formats + Excel serials, Latin-1 BR file, 6 SKUs omitted from the ERP master, mislabelled PSP fee column, snapshot-vs-ledger drift, negative sessions).
- **Staging** — 20 typed `stg_*` tables; every fix counted into DQ fragments.
- **Curated** — 16 dims + 12 facts (Parquet + CSV). Moving-average COGS from PO receipts; `fact_payment_schedule` explodes instalments; `fact_inventory_snapshot` derived from the movement ledger; USD conversion on every money column; `is_comparable_base` = US.
- **DQ gate** — all HARD checks pass: no null keys, full referential integrity, `net = gross - discount` , lines roll up to headers, payment schedule ties to order totals, comparable base = US, FX sane, volume/AOV/margin bands.

Calibrated over 4 regen passes. Final realised numbers: 294 SKUs · ~103k customers · ~158k orders · ~262k lines · AOV **\$208** · blended gross margin **14.2%** · return rate **7.8%** · MER **7.6×** · blended CAC **\$47** · repeat rate **40%** · inventory turnover ~5× · YoY **Total +140%** vs **Comparable-Base +36%**.

Step 5 — data documentation (done)

- [03_data_dictionary.md](#) — **auto-generated** by `etl/gen_data_dictionary.py` from the files on disk (raw + staging + curated, 1,186 lines).
- [04_etl_pipeline.md](#) — stage-by-stage narrative + techniques.
- [05_data_model.md](#) — bus matrix, fact grain & measures, dimension attributes, cross-cutting decisions, Power BI relationship list.
- [08_data_quality.md](#) — where issues are handled + the generated `data/quality/dq_report.md` + accepted limitations.

Step 6 — Power BI semantic model (done)

`etl/gen_semantic_model.py` generates the whole `.pbip` project from the curated Parquet schemas (model-as-code):

- `powerbi/VoltEdge Electronics.SemanticModel/` — TMDL: 28 data tables (16 dims + 12 facts) each with a `Parquet.Document` import partition driven by a `pDataFolder` parameter; `database.tmdl`, `model.tmdl` (auto date/time **off**), `expressions.tmdl`, `relationships.tmdl` (**65 relationships**, secondary data roles inactive), plus disconnected helpers `Reporting Currency` and `Funnel Stage`.
- `_Measures table` — **145 DAX measures** from `etl/pbi_measures.py`, in 14 display folders: Sales · Time Intelligence · Comparable Base · Targets · Reporting Currency · Payments & Cash · Price/Volume/Mix · Marketing · Website & Funnel · Logistics · Working Capital · CRM · Support & Returns.
- `powerbi/VoltEdge Electronics.Report/` — PBIR scaffold (7 pages, theme `powerbi/theme/VoltEdge.json`), verified: `pbir` loads the local model and validates **233 field references** with no errors.

Step 7 — Report pages (done)

`etl/gen_report_visuals.py` emits per-page `--from-json` bundles; applied with the `pbir` CLI. **7 pages, 93 visuals**: Executive Summary · Sales Performance · Marketing & Acquisition · Website & Digital · Logistics & Fulfillment · CRM & Customer · Product & Inventory. `pbir validate --all` → **Valid** (cosmetic warnings only: compact KPI cards, alt text, exec-page density). Layout and per-page rationale in [07_report_guide.md](#).

Step 8 — Measure & report docs (done)

- [06_dax_measures.md](#) — **auto-generated** from `pbi_measures.py`: every measure with its DAX and format string, grouped by display folder.
- [07_report_guide.md](#) — model overview, page-by-page walkthrough, the polish backlog, and how to verify in Desktop.

Step 9 — Visual design pass (done)

Stakeholder feedback: "visuals too raw (white on white, square corners), review the whole graphical layer, create and apply a company colour theme."

- **Design identity committed** — corporate tone, one accent (electric blue = current period, amber = prior year), muted neutrals, quiet gridlines. Signature: title band + right-side slicers + one 6-KPI row + two content rows on the detail gradient.
- **Comprehensive theme** `powerbi/theme/VoltEdge.json` — brand palette, semantic good/neutral/bad, six text classes, and `visualStyles` for 15 visual types + `page` (light `#F4F7FB` ground, white cards, 12 px radius, **no drop shadow**, axis titles off, value gridlines `#EEF1F6`, top-centre legends, clean tables). `pbir theme validate` passes.
- **Report rebuilt leaner** — canvas 1280×720; every page now = title + 2 slicers + 6 KPI cards + 4 content visuals (Exec has 5). **92 visuals, 0 overlaps**, `pbir validate --all` → Valid (warnings only: compact cards, one 4-series chart).
- **Measures fixed** — `New Customers` / `New Customer Revenue` / `Repeat Purchase Rate` now key off `dim_customer[customer_key]` so the new-vs-returning split propagates to `fact_order_lines`; `DATESYTD` year-end set to `"06-30"`.
- One command rebuilds the whole report: `cd powerbi && bash build_report.sh`.

Step 10 — Design pass 2 (stakeholder feedback)

Feedback after opening in Desktop: model loaded past the TMDL errors; some measures still error; "better, but needs more punch; cards have a duplicate label and feel cramped; put the date axes in ascending date order."

- **Duplicate KPI label removed** — `card.*.categoryLabels.show = false` (the visual title is the label); KPI value 24 → 32 pt; card height 84 → 100 px so the number breathes.
- **Header signature** — every page gets a `#EEF3FB` header zone with a `#1F6FEB` brand accent rule beneath it, title on the band. `shape` styling folded into the theme.
- **Date axes ordered** — `dim_date` now carries hidden `month_sort` / `quarter_sort` keys and `sortByColumn` on `month_year` / `month_name` / `quarter_name` / `day_name`; every time-series visual also gets an explicit ascending `pbir visuals sort`.
- **Measures** — `New Customers` / `New Customer Revenue` / `Repeat Purchase Rate` / `Customers with Orders` / `Active Customers` re-keyed to `dim_customer[customer_key]` so the filter propagates to `fact_order_lines`; `DATESYTD` → `"06-30"`.

`bash powerbi/build_report.sh` → theme valid, model loads, 215 fields resolve, no errors.

Step 11 — Design v2: deep blue + logo

Stakeholder: "better; go darker blue; create a company logo and put it top-left of every page."

- `etl/gen_logo.py` (Pillow) generates the VoltEdge mark — a lightning-bolt icon + "VOLT" / "EDGE" wordmark — as `powerbi/assets/volte-edge-logo-light.png` (for the navy bar), `-dark.png` and an icon-only variant.
- **Theme v2** — deep blue `#123E6E` primary, navy `#0E2A4E` header, amber `#D99311` comparison, mid-blue `#2E70B0` second series; cooler `#EAEFF6` page ground; KPI value 32 pt deep blue.
- **Header** — every page now has a 56 px navy bar with the logo top-left and a `#2E70B0` accent rule; the page title and the two slicers sit on the line below.
- `bash powerbi/build_report.sh` → theme valid, model loads, 215 fields, **no errors, no real content overlaps** (only the expected decorative header-band overlaps).

Step 12 — v3: header image, combo charts, date filter, richer seasonality

Stakeholder: logo not rendering; title poorly placed (wants it on the navy bar in a light colour); dual-axis for discrepant-scale chart pairs; a date filter to pick the analysis base month; and the data is too smooth — add seasonality to revenue, cash, sessions, and new-vs-returning (so those lines cross more than once); the funnel's 500M→9M drop is extreme.

- **Header is now one image per page** (`etl/gen_logo.py` → `assets/header-<slug>.png`): navy bar with the white bolt + `VOLTEEDGE` wordmark + divider + page title + a `#2E70B0` accent rule. Fixes the un-rendered image visual *and* puts a light-on-dark title on the bar. The band shapes + separate logo + title textbox are gone.
- **Combo charts** — every "big number + ratio" pair (sessions/CR, delivery-days/on-time, spend/ROAS, revenue/margin%, inventory-value/weeks-of-cover, RPS/CR) is now a `lineClusteredColumnComboChart`: the magnitude as bars, the ratio as a line on a secondary axis.
- **Analysis-period slicer** — a date-range slicer (`dim_date[date]`) on every page, so KPIs and point-in-time measures (inventory, receivables) are "as of" the chosen range end rather than always the latest data.
- **Data seasonality** (`etl/config.py`, `etl/01_generate_raw.py`): deeper monthly seasonality curve, a slow demand wave, $\pm 10\%$ month noise, and occasional hot/soft months; `RETURNING_ORDER_SHARE` now oscillates so new- and returning-customer revenue weave and cross several times. Cash-received / revenue-booked and sessions inherit the shape from orders.
- **Funnel** rebuilt as the **on-site funnel** — Sessions → Add-to-Cart → Cart → Orders (sane $\sim 100/12/8/2$ ratios). Media metrics (impressions, clicks, CTR) stay as cards.

Step 13 — v4: profitability logic, budget everywhere, chart re-evaluation

Stakeholder feedback (contribution margin too negative, illegible filters, "vs budget" on every KPI, add Orders to Sales, more delivery variance, explain booked vs cash, is 100 % of new revenue from marketing?, inventory chart only shows one month, re-evaluate every chart, grow the page if needed).

Data / model - Returns COGS double-count fixed — `fact_returns` now carries `cogs_recovered_usd` (restocked → full moving-avg cost back, scrapped → 25 % salvage); `Gross Profit` credits it. $\sim \$2$ M swing. Blended gross margin 14 % → **19 %**, contribution margin -18% → **-4 %** (near break-even in normal months, dips in the Q4 promo pushes — the CM bridge shows why). - Category margins +5 pp, `TARGET_MER` 8.5 → 10. - **Booked vs cash gap made real** — an 8 % processor **rolling reserve** released at +90 d on card/wallet/BNPL volume, on top of settlement lag and BR instalments. Cumulative gap $\approx 5-7\%$ of revenue, now visible. - **Delivery variance** — smooth monthly logistics-stress (AR-1) + carrier-crisis months + a heavy/oversized-item effect. Avg delivery days ≈ 5.6 (range 4.3–8.5); on-time % 81 % (range 38–95 %). - New measures: `Returns COGS Recovered`, `CM Bridge Value` (+ PL Line helper table), `Revenue Booked/Cash Received (Cumulative)`, `Booked - Cash Gap`, `Inventory Value / Weeks of Cover (Month-End)`, `% New Customers via Paid`, ... vs `Target %` for Orders / New Customers / Blended CAC / Gross Margin %.

Report - Canvas **1280 × 860**. Header stays a single navy image; **slicer row** with a wide date-range slicer + Currency/Market as **horizontal tiles** (readable). - **KPI tiles are kpi visuals** (value + plan Goal + trend sparkline) wherever a target exists; `card` otherwise. Executive Summary carries a **Budget scorecard** table. - **Contribution margin bridge** waterfall replaces the flat CM-% line. - **Every chart re-evaluated** (see `07_report_guide.md`): Sales gets Orders + cumulative booked-vs-cash; Marketing gets *New customers by acquisition channel* + on-site funnel; Product & Inventory swaps the revenue×margin combo for *margin % by category + return rate % by category*, and the inventory trend uses month-end measures so **all** months render.

`bash powerbi/build_report.sh` → theme valid, model loads, 244 fields resolve, no errors, no real overlaps.

Step 14 — v5: fixes to the fixes

Stakeholder feedback on v4 (budget scorecard broken; PY missing on the exec trend; Nov CM crater; want a revenue-vs-plan bridge too; booked-vs-cash should be monthly and the gap bigger; want vs-plan by category on Sales; want an AI-fed insights section; campaign orders/revenue nonsensical; delivery-days range still too wide; cards should show only the % vs plan, not the target number — all in English).

- **Budget scorecard** — was repeating the grand total on every row (`dim_metric` on rows + measures that don't vary by metric). Rebuilt with `Budget Actual / Budget Target / Budget Attainment %` — `SWITCH` on the row's metric; CAC scored as lower-is-better.
- **KPI cards** — dropped the `kpi` visual (it forces the goal number). Back to `card` with two values: the metric + its ... vs `Target %`. English throughout.
- **Exec net-revenue trend** — visual filter to `date ≥ 2025-01`, so the prior-year overlay covers most of the plot (PY only exists from 2025-07 in a 24-month extract).
- **Nov CM crater** — in promo months the generator now **pulls marketing back ~30 %** (demand is already there) and promo discount depth is $\sim 25\%$ lighter. Monthly CM % went from $\sim -14\%$ in the Q4 pushes to $\sim -7\%$, most months -1 to -3% ; blended -2% . New **Contribution Margin % (Trailing 12M)** line on the Exec page shows the improving, toward-break-even trend without the Q4 spikes.
- **Revenue-vs-plan bridge** — new `Revenue Bridge` helper table + `Rev Bridge Value`: waterfall Plan → $\pm$ each market's variance → Actual, next to the CM bridge on Exec.

- **Booked vs cash** — back to **monthly** (not cumulative); processor rolling reserve 8 % → **14 %**, released at 75 d; card settlement +2 d. The monthly gap now swings ±10–25 % (peak months collect less than they book; trough months collect the reserve/installment tail). Shown as clustered bars + a receivables-outstanding line.
- **Sales — revenue vs plan by category** — `fact_target` has no category grain, so a new measure allocates the month/market Net Revenue target to categories by their revenue share; waterfall on the Sales page.
- **AI insights** — a native **AI Narratives (Smart Narrative)** visual on the Exec page: auto-generated summary that updates with the date slicer and every filter, ships in the `.pbip`, no external key. Productizable as-is.
- **Campaign table** — orders/revenue aren't campaign-attributed in the data (no order→ campaign link; would need MTA). Table now shows media metrics only — Spend, Impressions, Clicks, CTR %, CPC. Limitation documented.
- **Delivery variance** — logistics-stress amplitude cut, carrier-crisis spikes made mild, bulky-item effect shrunk, carrier base days compressed. Avg delivery days ≈ 5.5 (monthly 4.6–7.5, was 4.3–10.9).
- **Buffer months** — `in_extract_window = true` page filter on every page removes the pre-window `dim_date` rows from the axes.
- Canvas **1280 × 900** to fit the narrative band.

`bash powerbi/build_report.sh` → theme valid, model loads, 265 fields resolve, no errors, no real overlaps.

Step 15 — v6: broken filter, header, and a real profitability trajectory

v5 opened with several visuals broken. Causes and fixes:

- **`in_extract_window` page filter broke every page** (boolean column + a string `"true"` categorical value → "Algo está errado com um ou mais filtros", cascading to all visuals). Replaced with a string calc column `dim_date[period_label]` ("Analysis window" / "Buffer") filtered categorically — safe.
- **Header images not rendering** — `imageScaling.imageScalingType` was `Normal`; set to `Fit` on every header image and in the theme.
- **Budget scorecard** — one table column can't format \$ + count + % rows. Added `Budget Actual (fmt) / Budget Target (fmt)` string measures that `FORMAT` per row; table total turned off.
- **CM "improving" was invisible because it wasn't improving** — real trailing-12M CM % was flat at ~-2.5 %. Engineered a genuine trajectory in the data: **marketing efficiency ramps over the window** (`TARGET_MER` 6.5× → 14×, so spend falls from ~15 % to ~7.6 % of revenue) and **unit costs drift down ~0.4 %/mo** (procurement scale economies). Result: monthly CM % **-6.5 % (Jul-24) → +4.2 % (Jun-26)**; trailing-12M **-3.4 % → +1.9 %**; blended ≈ break-even. The Exec CM chart is now a combo — monthly bars + the trailing-12-month line so the upward trend is unmistakable.
- **Delivery days** — `PEAK_DELAY_INFLATION` 1.6 → 1.22, promised buffer +1 d: monthly avg now **4.7–6.3 d** (was 4.6–10.9); on-time 91–100 %.
- Removed the `eff` reference left dangling in `gen_marketing` (had halted stage 01).

`bash powerbi/build_report.sh` → theme valid, model loads, 266 fields resolve, no errors, no real overlaps.

Step 16 — v7: kill the fragile bits

v6 still had broken visuals. Root causes were the workarounds themselves:

- **The page filters I added broke the pages.** `dim_date[period_label]` (a calc column) as a page filter and `dim_date[date] ≥ 2025-01` on `exec_trend` both threw "Algo está errado com um ou mais filtros", cascading to the KPI cards. **Removed every added filter.** The pre-window buffer months are not actually an axis problem (the time-series visuals join on `order_date_key`, which only has in-window rows).
- **The image-header approach is unreliable through `pbir`** on a hand-generated PBIR — Desktop shows "no image selected" even with the resource registered. Replaced with a plain **shape header strip** (`#E7EDF7`) + a `#123E6E` accent rule + a `pbir add title` textbox (dark text on the light strip). No image resources, always renders.
- **AI Narratives visual needs a manual "Personalizado" click** to activate (or Copilot). Replaced with a **dynamic insight card** bound to a new `[Exec Insight]` text measure: a one-paragraph summary (net revenue, YoY total + like-for-like, vs plan, gross & contribution margin, trailing-12m CM and its verdict, CAC, repeat rate, on-time) that **updates with the date slicer and every filter**, no interaction, ships in the file. Smart Narrative remains available for anyone who wants to add it.
- **Build hardened** — `build_report.sh` now resets and retries a page if a OneDrive/AV file lock corrupts its bundle mid-write (this had left the Sales page half-built).

`bash powerbi/build_report.sh` → theme valid, model loads, 244 fields resolve, no errors, no real overlaps (only the header shapes sit behind the title, by design).

Step 17 — v8: the real root causes

v7's "fixes" were built on a broken toolchain. Found and removed the actual causes.

- **`pbir` 0.9.29 was silently corrupting every multi-measure KPI card.** Each card that shows a value plus its "vs plan %" got an *empty* `Advanced` filter injected per measure. An empty filter → "Algo está errado com um ou mais filtros" → the card renders as a dead "Ver detalhes" tile. That was the "visuais quebrados". Upgraded to **`pbir` 0.9.31**, which builds these cards clean. Verified: no `filterConfig` on any KPI card now.
- **`pbir` 0.9.31 needs the .NET ADOMD client** (it now schema-checks every field binding). Not on PATH here → `add visual --from-json` threw `TypeError: AdomdConnection` and added *zero* visuals. `build_report.sh` now exports `PBIR_ADOMD_DIR="/c/Program Files/Power BI Report Builder"`.
- **Header is a real image again.** Reverted the v7 shape-strip hack. Each page gets `assets/header-<page>.png` as an `image` visual across the top (x0 y0 1280×60); the PNG already carries the bolt + VOLTEDGE + page title + rule. 0.9.31 registers the resource in `report.json` correctly — the

earlier "no image selected" was a 0.9.29 bug.

- **Date slicer is usable.** Was sitting on the raw `date` column with a stray empty filter and no control. Now a **Between range slider** ("Analysis period"); currency / market are horizontal tiles. Header row grew to fit (slicers y72 h64, KPIs y148, content starts y352/y556). Removed "(trailing 18 months)" from the revenue chart title.
- **Delivery days back in the 4-5 band.** `PEAK_DELAY_INFLATION` 1.22 → 1.06 (killed the Q4 spike to ~7), tighter logistics-stress band, transit σ 0.8 → 0.6. Result: every month 4.5–4.9, overall 4.67.
- **On-time delivery engineered to ~93%.** With the calmer transit, nothing was ever late (flat 99–100%). Added `SLIGHTLY_LATE_SHARE = 0.055`: a seeded slice of orders has its *promise* pulled in so the (unchanged) delivery lands 1–2 days past it. Result: on-time 92–94% by month (overall 93.3%), ~6.7% of orders 1–2 days late; average delivery time stays 4.5–5.1 days (overall 4.81).
- **pbir 0.9.31 mangles – in visual titles** (writes `â€`) and its `--from-json` field resolver won't bind a measure whose name contains `" - "`. So every report title and ~12 measure names were made fully ASCII (`Funnel – Value` → `Funnel Value`, `PVM – Price Effect` → `PVM Price Effect`, `CLV – Gross Profit` → `CLV Gross Profit`, `Booked – Cash Gap` → `Booked vs Cash Gap`, etc.). Field bindings themselves keep working; it is only the title literal and the `" - "` separator that break.
- **pbir 0.9.31 validates add visual fields against the running Desktop's live model, not the TMDL on disk.** With Desktop open on the report, every renamed/added measure reads as "not found in model" and the page builds with only its header. `build_report.sh` now aborts if `PBIDesktop.exe` / `msmdsrv.exe` is running — close Desktop, build, reopen.

`bash powerbi/build_report.sh` (Desktop closed) → 7 pages, 103 visuals, 244 fields, 0 errors, 67 advisory warnings (dense KPI tiles + short slicers, all by design). Header images registered + referenced on all 7 pages.

Step 18 — v9: verified against the running canvas

With the Desktop bridge on, screenshotted all 7 pages and fixed what the JSON couldn't show.

- **The header image visual never renders a bound RegisteredResource through pbir** — Desktop shows "select an image". A **page background image** does render. Header is now a full-canvas `pagebg-<slug>.png` (1280×900: navy brand band on top, `#EAEFF6` below) set via `pbir pages background . gen_logo.py` emits it; `build_report.sh` applies it and the `image` visual is gone.
- **The classic card visual shows exactly one field.** The "value + vs-plan %" cards were the "Ver detalhes" tiles all along (not the 0.9.29 filter bug — that was real but separate). Live fix: KPI cards reverted to single-measure (all 7 pages render clean). Generator fix for the next rebuild: each KPI tile is a value card **plus a small "vs plan" delta card** beneath it where a plan target exists; KPI band height re-flowed (value h58 @ y148, delta h34 @ y208, content from y250).
- **kpi visual type rejected** — it shows the *last* trend point, not the period total (\$1.9M not \$29.5M), and went blank for CAC / New Customers. Not used.
- **Grand-total rows** removed from the four `tableEx` visuals (`total.totals = false`).
- **Narrow monthly combo charts scrolled** (24 bars in a 4-col slot). `sal_cash` and `mkt_trend` widened to 8 cols in the generator; `sal_planbridge` dropped to make room (revenue-vs-plan is still on the exec page, by market).
- **Stale data in Desktop.** `.SemanticModel/.pbi/cache.abf` is Power BI's imported-data snapshot; reopening the `.pbip` reuses it instead of re-reading the current parquet, so the canvas still showed the pre-fix delivery numbers. `build_report.sh` now deletes `cache.abf`; until a rebuild, hit **Atualizar** in Desktop to re-import.

Live report (via `pbir` mutations, no rebuild): headers on all 7 pages, KPI rows clean, table totals gone, chronological sorts, exec Budget scorecard trimmed to metric / Actual / Attainment % (this is the "vs budget" until the delta cards land), `crm_cohort` filtered to the 24 in-window months. After the stakeholder hit **Atualizar** the fresh data showed: delivery 4.8 d/mo (no Q4 spike), on-time 93.9% (waving 93-95%), gross margin 18.3%, contribution margin at break-even, CM% trend visibly rising.

Known, deferred to the rebuild / ETL: - `dim_date[month_year]` surfaces a blank "(Em branco)" member on `log_cash` (a fact date key falls outside the 2024-2026 `dim_date` span — `Cash Received` settling past 2026-06). Fix in ETL: widen `DIM_DATE_END` or clamp payment/PO date keys. - `sal_cash` / `mkt_trend` still narrow with a scrollbar (generator widens them to 8 cols). - KPI cards have no inline vs-plan yet (generator adds the delta sub-cards).

Step 19 — v10: stakeholder review round

Screenshot review with the stakeholder. Points raised and how each was handled:

- **Slicers showed no options.** The row was 64 px tall — a Between slider and Dropdown buttons need more. Page canvas grown to **1280 × 1000**; slicer row is now 92 px, KPI row 72 px, content rows re-flowed. Date slicer = `Between`, currency/market = `Dropdown`.
- **KPI values were clipped.** Value cards back to a real height (72), value type 30 pt; the "vs plan %" delta card sits underneath (28 px) where a plan target exists.
- **" +44% YoY but –10% vs plan — does that make sense?"** It didn't: the old plan was `actual ÷ attainment(0.9–1.1)` every month — a plan with no link to prior-year growth, effectively "actual + 11%". Reworked `gen_targets`: current-year months are planned as the **prior-year actual grown by `PLAN_YOY_GROWTH` (40%)**; pre-CY months sit near actual. So a +44% YoY delivery now lands a few points **ahead** of a 40%-growth plan, with monthly attainment swinging on `PLAN_MONTH_NOISE`.
- **Data labels.** Added at theme level for bar/column/line/combo/waterfall (9 pt); the five dense 24-month time-series keep labels off so the axis stays readable.
- **"Explain the logistics working-capital chart."** It plots the Cash Conversion Cycle by month — **DIO** (days of stock held) + **DSO** (days from sale to cash) – **DPO** (days taken to pay suppliers) = **CCC** (net days cash is tied up). It was mis-scaled: DIO/DPO divided a one-month inventory/payables balance by a *trailing-12-month* COGS, so in the first year (window not yet full) the denominator was tiny and the bars blew up to ~3,000 "days". Fixed to a period-consistent formula (`balance × days-in-period ÷ COGS-of-that-period`). Also cut `TARGET_WEEKS_OF_COVER` 5 → 3 so inventory (and therefore DIO) sits in a believable band.

Next

- [] `bash powerbi/build_report.sh` → **open the .pbip** (Desktop already closed) for the v10 layout + rebased plan + fixed CCC measures + fresh data.
- [] ETL: kill the blank `dim_date` member feeding `log_cash`.
- [] Alt text pass; trim curated CSVs; finalise README; sync to Google Drive.
- [] Step 5 — `02_clean_stage.py`, `03_build_curated.py`, `dq_checks.py`, run pipeline
- [] Step 6 — data dictionary, ETL doc, data model doc, DQ doc
- [] Step 7 — Power BI semantic model + measures + 3 pages + theme
- [] Step 8 — DAX measures doc, report guide
- [] Step 9 — finalize walkthrough & README, sync to Google Drive